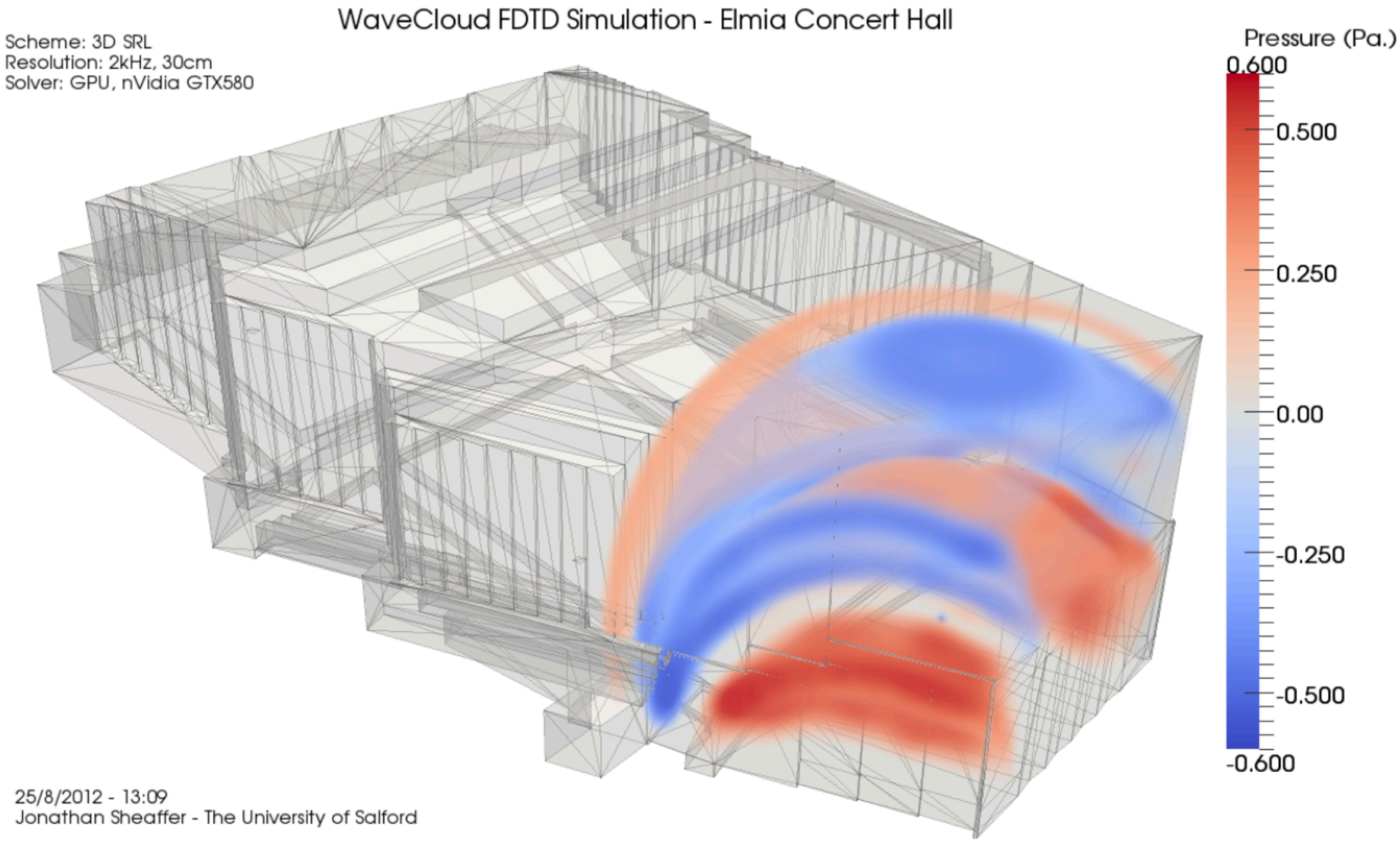
Jet marching methods for numerical geometric acoustics

Samuel F. Potter

Courant Institute of Mathematical Sciences

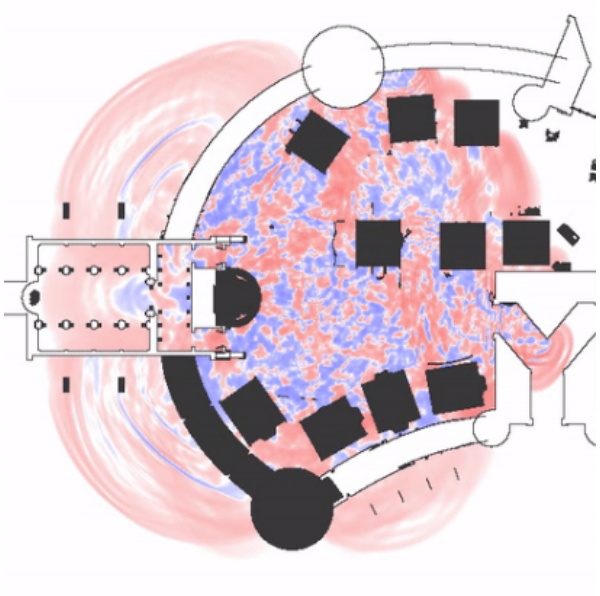
Introduction

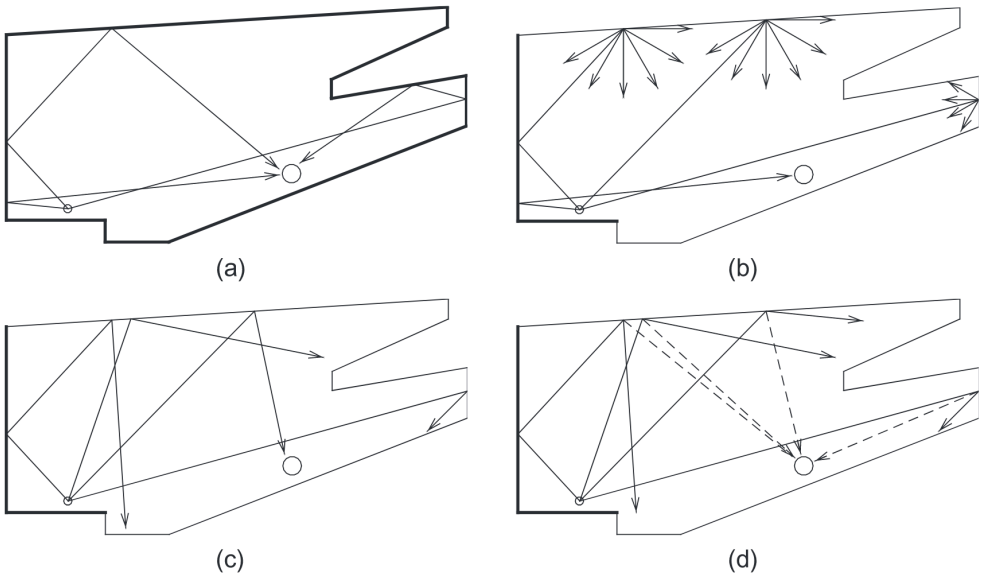
Introduction: computational room acoustics



Sheaffer, 2013.

Introduction: computational room acoustics





Numerical acoustics: solve wave equation for bandlimited impulse input. High-frequency (>2kHz) simulations expensive and impractical.

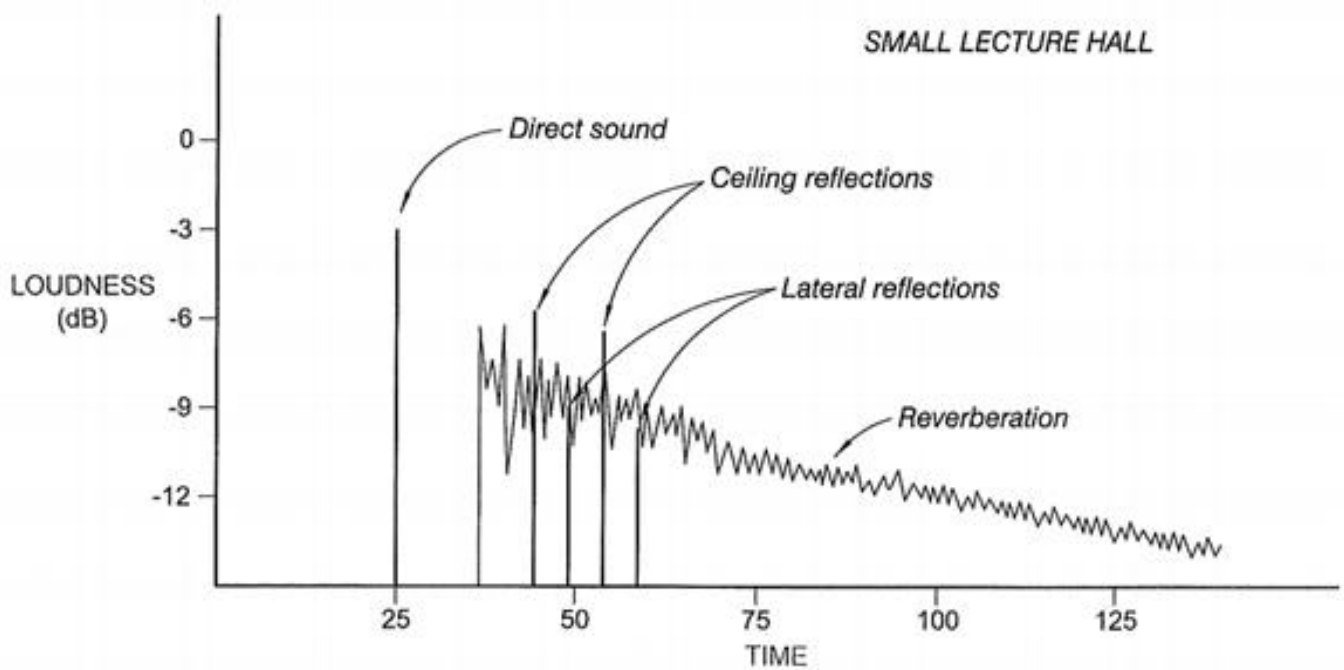
Geometric acoustics: use raytracing to assemble room impulse response. Handling diffraction effects with raytracing is complicated.

Left: `https://www.microsoft.com/en-us/research/project/project-triton`

Right: Kuttruff, 2016.

Introduction: precomputed sound propagation

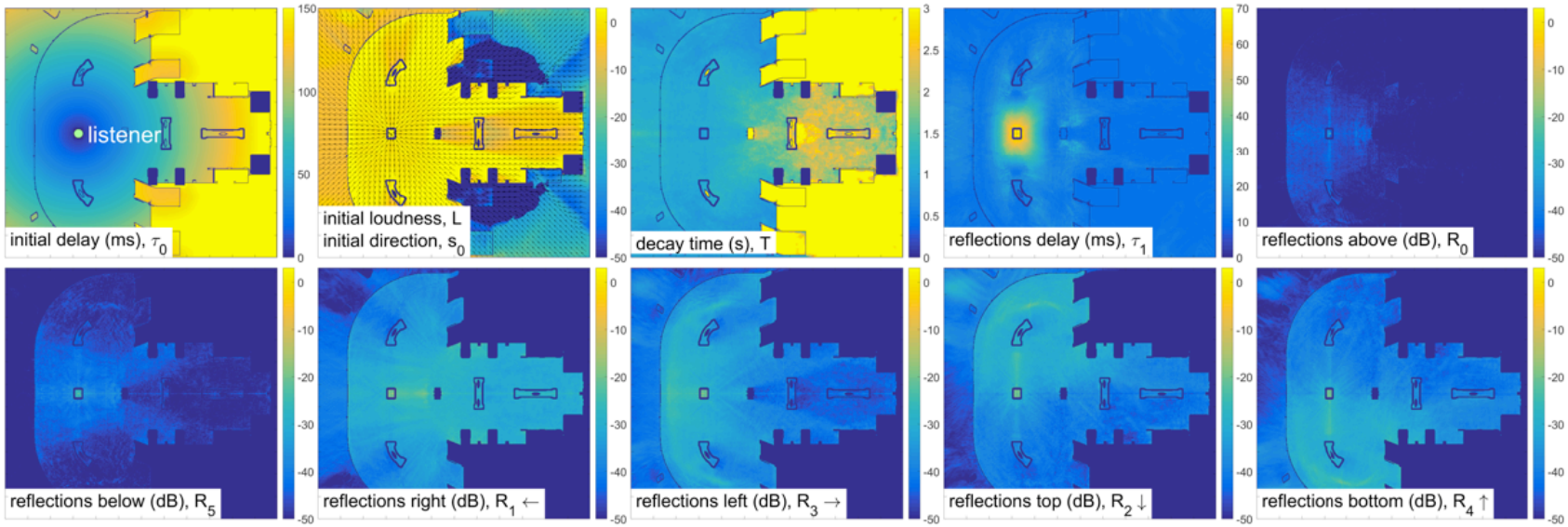
Basic task: for a pair of points in the room (a *source* and *listener*), compute or encode the room impulse response. This is the Green's function for the wave equation for a *fixed sound source location*.



Source: `https://sites.google.com/site/roomacgroup4/`

Introduction: precomputed sound propagation

Too expensive to store the room impulse response for each source location. Instead, extract smooth fields containing **important perceptual parameters** and compress these.



Source: `https://www.microsoft.com/en-us/research/project/project-triton`

Introduction: PDEs for acoustics

Wave equation for pressure $p = p(\boldsymbol{x}, t)$:

$$p_{tt} = c^2 \Delta p$$

Time-harmonic solution for frequency ω:

$$p(\boldsymbol{x}, t) = e^{i\omega t} u_\omega(\boldsymbol{x})$$

Leads to the Helmholtz equation:

$$\Delta u_\omega + k^2 u_\omega = 0, \qquad k = \frac{\omega}{c}$$

where c is the speed of sound and k is the wavenumber.

Introduction: high-frequency wave propagation

Geometric optics ansatz ("WKB") for Helmholtz:

$$u_\omega(\boldsymbol{x}) \sim A_\omega(\boldsymbol{x}) e^{i\omega\tau(\boldsymbol{x})}$$

Leads to the eikonal equation:

$$\|\nabla\tau\| = 1/c = s, \qquad s \text{ is the "slowness"}$$

and the amplitude equation:

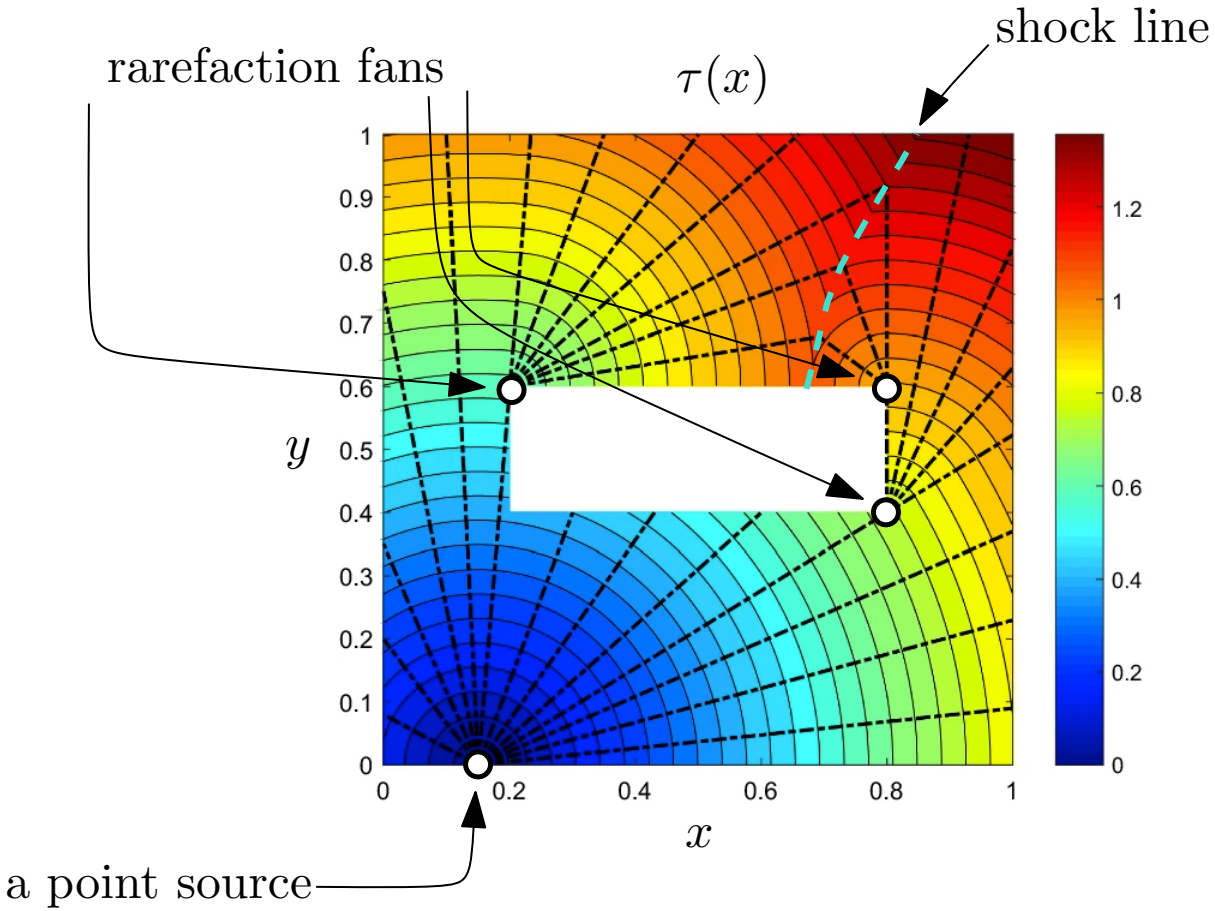
$$\nabla A_\omega \cdot \nabla\tau + 2A_\omega \Delta\tau = 0$$

High-frequency approximation of the Helmholtz equation:

$$|u_\omega - A_\omega e^{i\omega\tau}| = O(\omega^{-1})$$

Introduction: the eikonal equation, $\|\nabla\tau\| = 1/c$

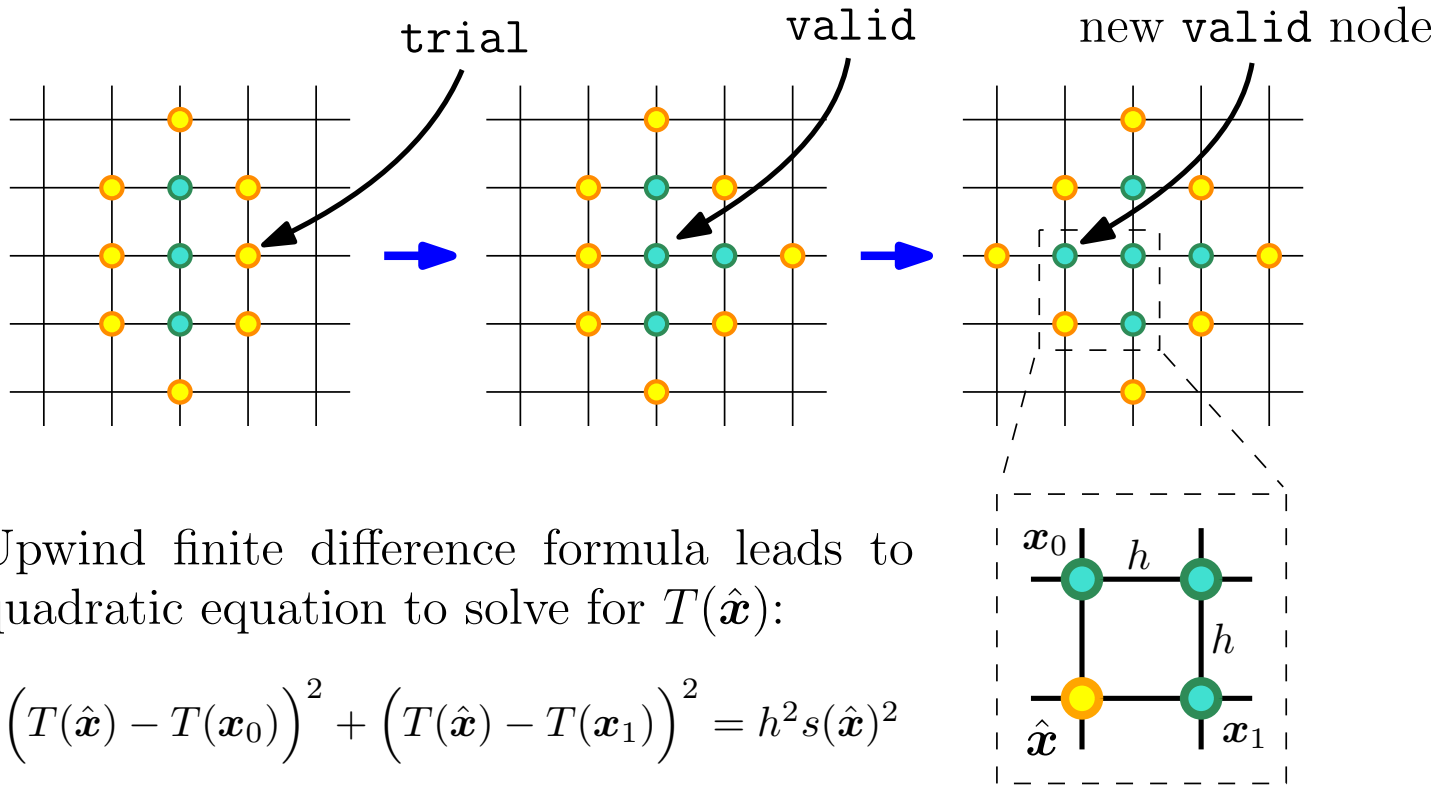
A PDE whose solution encodes only the first arrival time accounting for diffraction. Its characteristics are rays.



Source: Qi & Vladimirsky, 2019.

Introduction: the fast marching method

A single-pass, $O(h)$ accurate, upwind finite difference method for solving the eikonal equation based on Dijkstra's algorithm



Upwind finite difference formula leads to quadratic equation to solve for $T(\hat{\boldsymbol{x}})$:

$$\Big(T(\hat{\boldsymbol{x}}) - T(\boldsymbol{x}_0)\Big)^2 + \Big(T(\hat{\boldsymbol{x}}) - T(\boldsymbol{x}_1)\Big)^2 = h^2 s(\hat{\boldsymbol{x}})^2$$

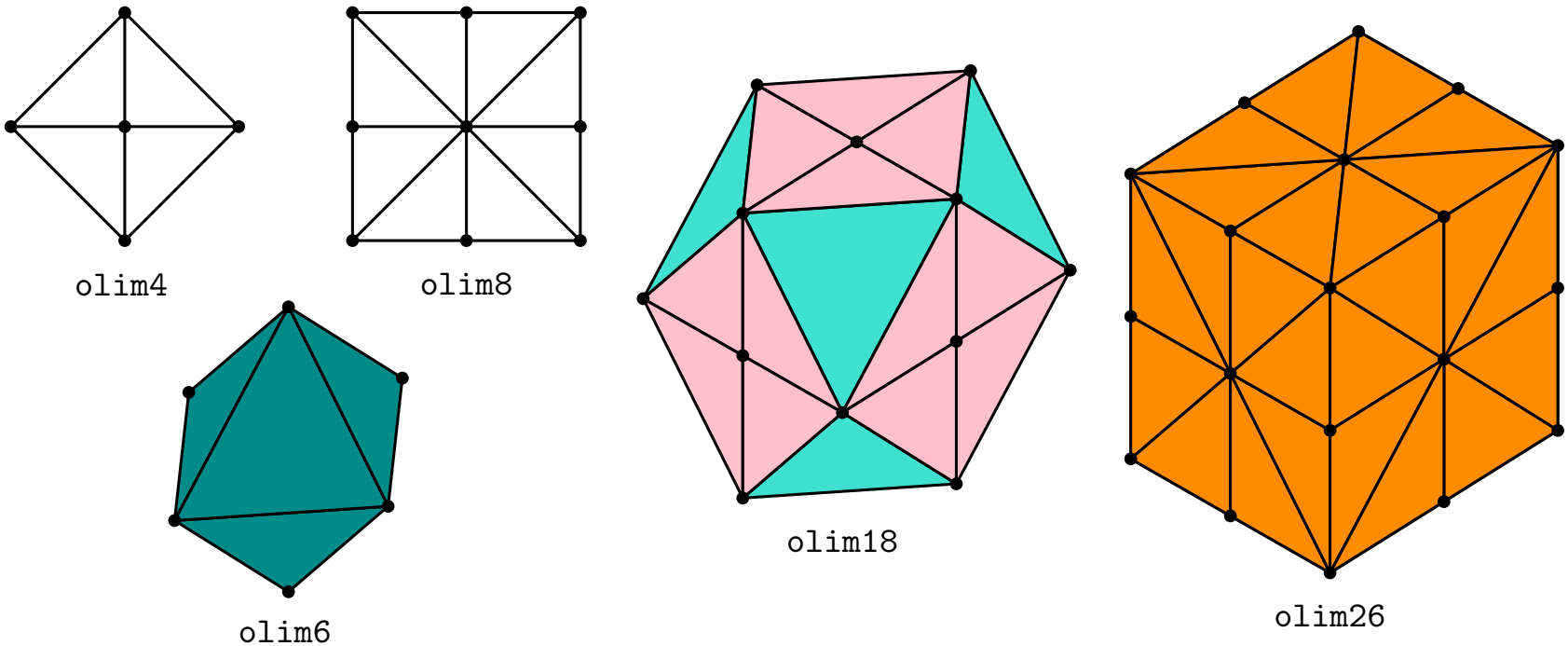
All `trial` nodes ordered in a priority queue by current T value

Sethian, 1996.

Introduction: semi-Lagrangian marchers

Can also solve the eikonal equation by combining local raytracing (semi-Lagrangian) with marching (Dijkstra). Ex: Tsitsiklis's algorithm, **OLIMs**...

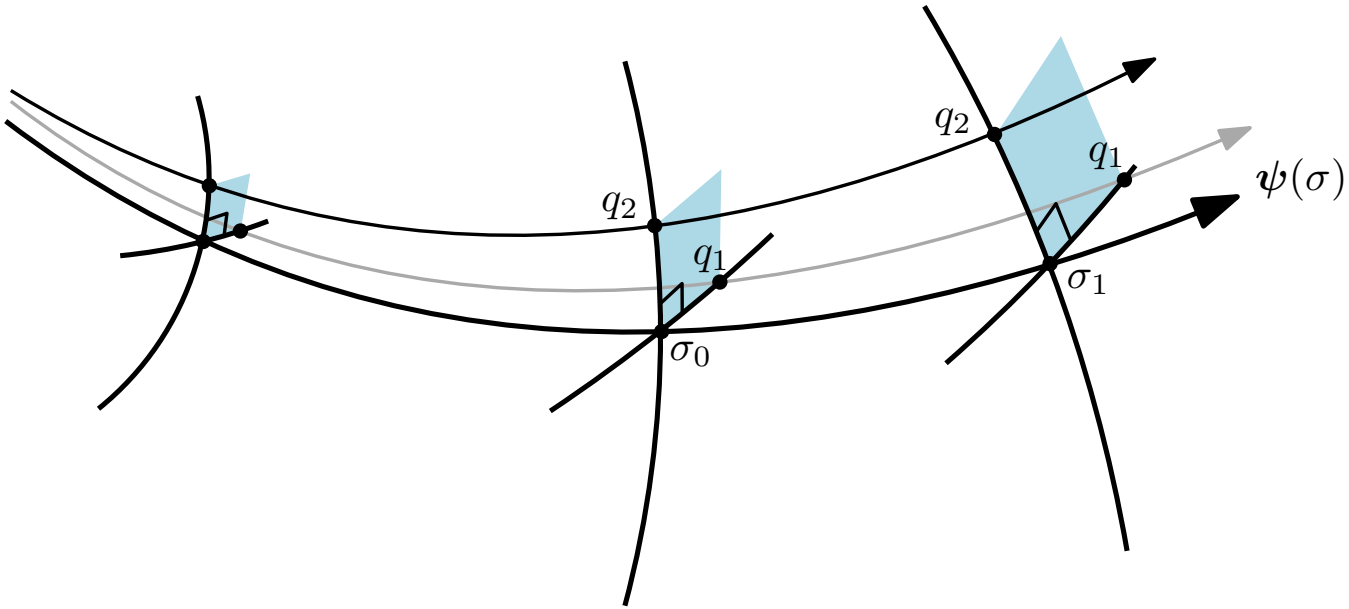
$$T(\hat{\boldsymbol{x}}) = \min_{\boldsymbol{\psi}} \left\{ T(\boldsymbol{\psi}_0) + \int_{\boldsymbol{\psi}} s d\ell \right\}$$



Potter and Cameron, 2019.

Introduction: the amplitude equation

Using **paraxial raytracing**, the amplitude can be computed in terms of the local geometric spreading of a ray tube:



The amplitude along a ray ψ satisfies:

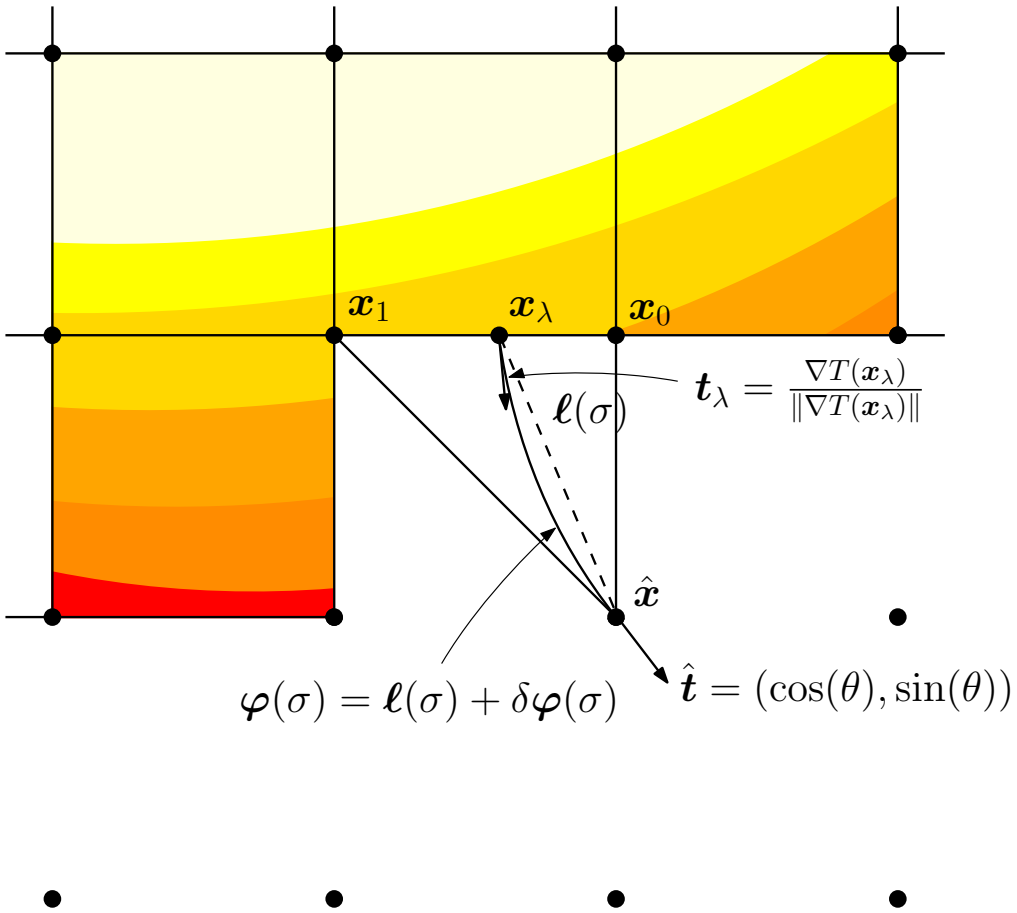
$$\alpha(\psi(\sigma_1)) = \alpha(\psi(\sigma_0))\sqrt{\frac{s(\psi(\sigma_1))}{|\det(Q(\sigma_1))|}}, \qquad Q = \begin{bmatrix} q_1 & q_2 \end{bmatrix}$$

Popov, 1978.

Jet marching methods (JMMs) in 2D

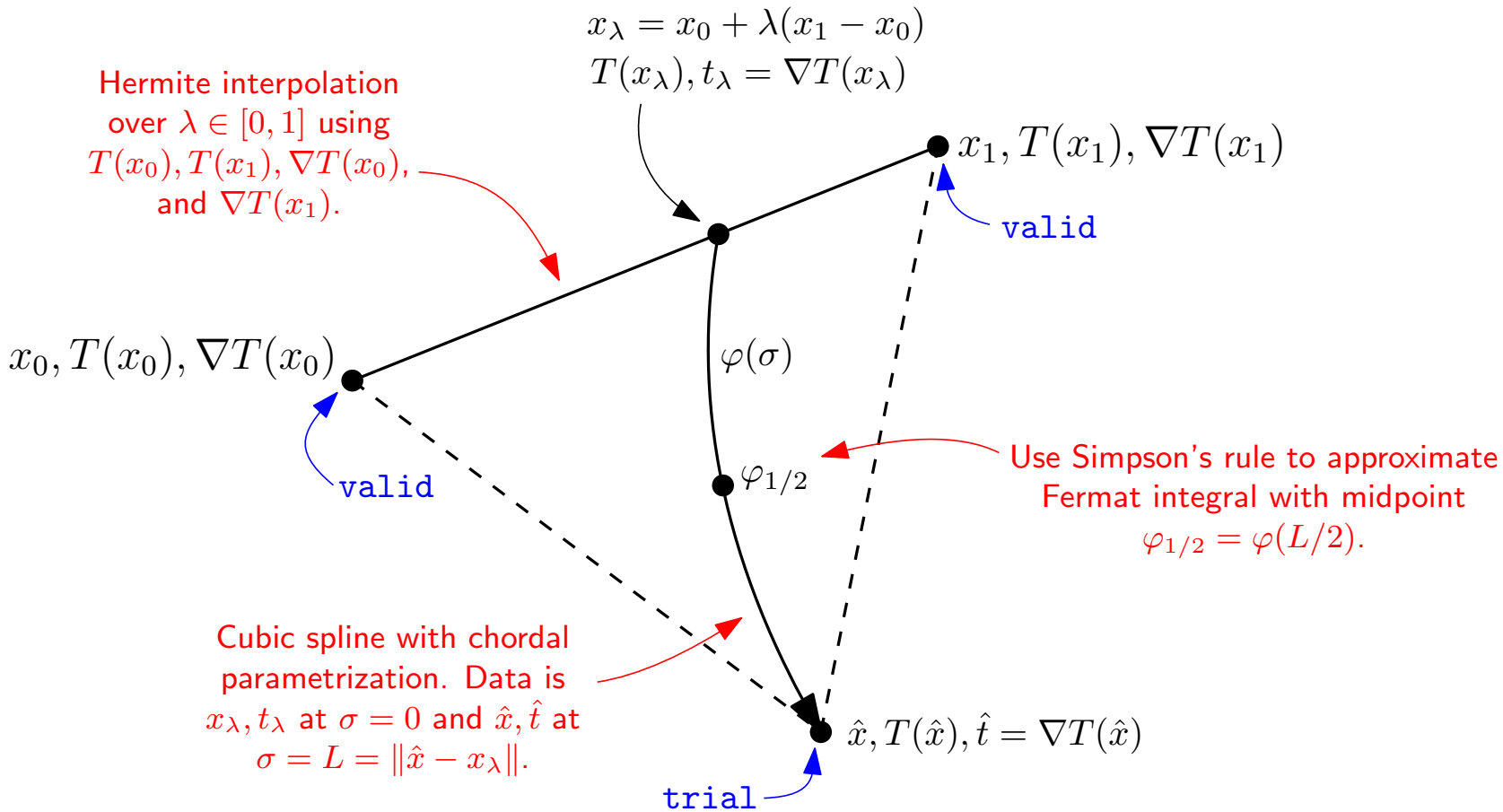
2D JMMs: overview

Approximate T and ψ using cubic polynomials, march the **jet** of the eikonal (coefficients of truncated Taylor polynomial):



Potter and Cameron, 2021.

2D JMMs: higher-order updates in 2D



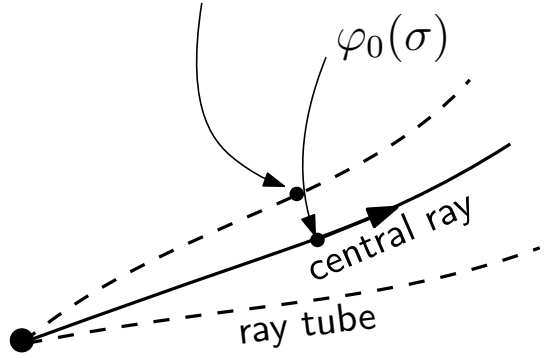
$$T(\hat{x}) = \min_{\substack{\hat{t}\in\mathbb{S}^1 \\ 0\le\lambda\le 1}} \left\{ \mathsf{T}(\lambda) + \frac{L}{6}\left(s(\hat{x}) + 4s(\varphi(\tfrac{L}{2}))\|\varphi'(\tfrac{L}{2})\| + s(x_\lambda) \right) \right\}$$

Potter and Cameron, 2021.

2D JMMs: marching the geometric spreading

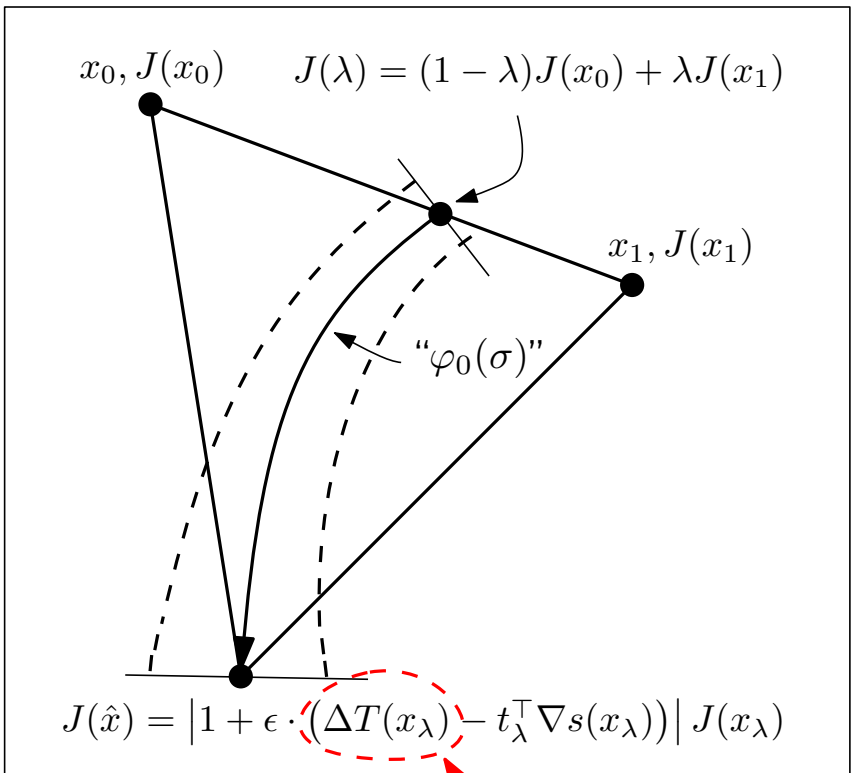
Linearly approximate the geometric spreading J over $[\boldsymbol{x}_0, \boldsymbol{x}_1]$ and march it after parametrizing the local ray:

$$\varphi(\sigma, q) = \varphi_0(\sigma) + E(\sigma)q$$
$$E^\top \varphi_0' \equiv 0$$



$$\begin{bmatrix} \dot{Q} \\ \dot{P} \end{bmatrix} = \begin{bmatrix} & c_0 I \\ \frac{-1}{c_0} \frac{\partial^2 c}{\partial q \partial q^\top}\Big|_{q=0} & \end{bmatrix} \begin{bmatrix} Q \\ P \end{bmatrix}$$

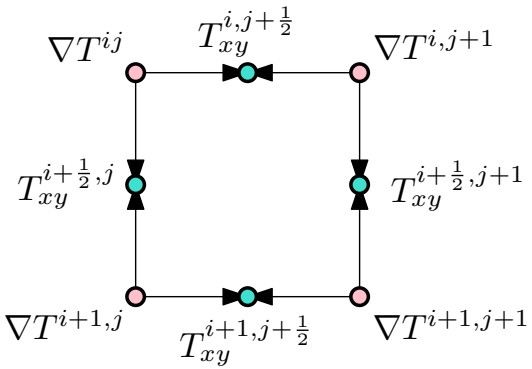
$$J = |\det Q|, \quad A(\phi_0(\sigma)) = \sqrt{\frac{c(\phi_0(\sigma))}{J(\phi_0(\sigma))}} A(\phi_0(0))$$



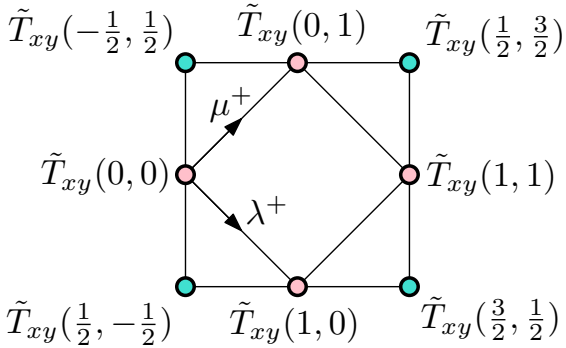
Potter and Cameron, 2021.

2D JMMs: cell marching

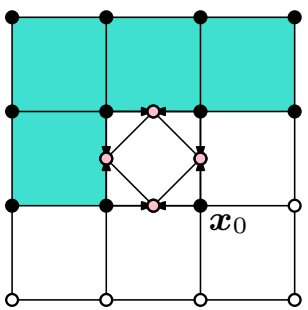
One way to march second partials (interpolation trick borrowed from Seibold et al., "Jet Schemes for Advection Problems"):



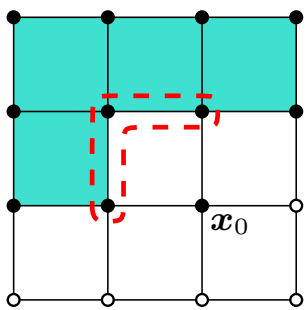
approximate T_{xy} at cell edge midpoints using central differences



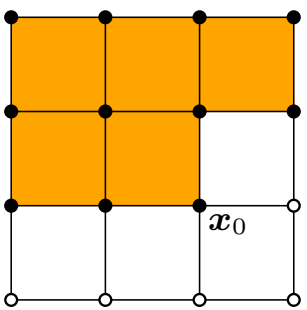
approximate T_{xy} at cell vertices using bilinear extrapolation



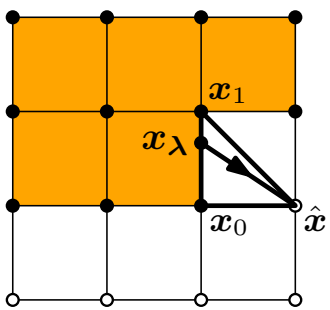
compute new T_{xy} values at corners of new `valid` cell



average T_{xy} values over adjacent `valid` cells



rebuild bicubics on affected `valid` cells



update neighboring `trial` nodes using triangle updates

Potter and Cameron, 2021.

2D JMMs: BCs for the geometric spreading

We march the geometric spreading using:

$$J(\hat{\boldsymbol{x}}) = \left|1 + \epsilon \cdot \left(\Delta T(\boldsymbol{x}_\lambda) - \boldsymbol{t}_\lambda \cdot \nabla s(\boldsymbol{x}_\lambda)\right)\right| \mathrm{J}(\boldsymbol{x}_\lambda),$$

where $\epsilon = \int_0^L c(\boldsymbol{\psi}(\sigma))d\sigma$. We need BCs for J for a point source:

$$J(\boldsymbol{x}) \sim \|\boldsymbol{x}\| \text{ as } \boldsymbol{x} \to 0.$$

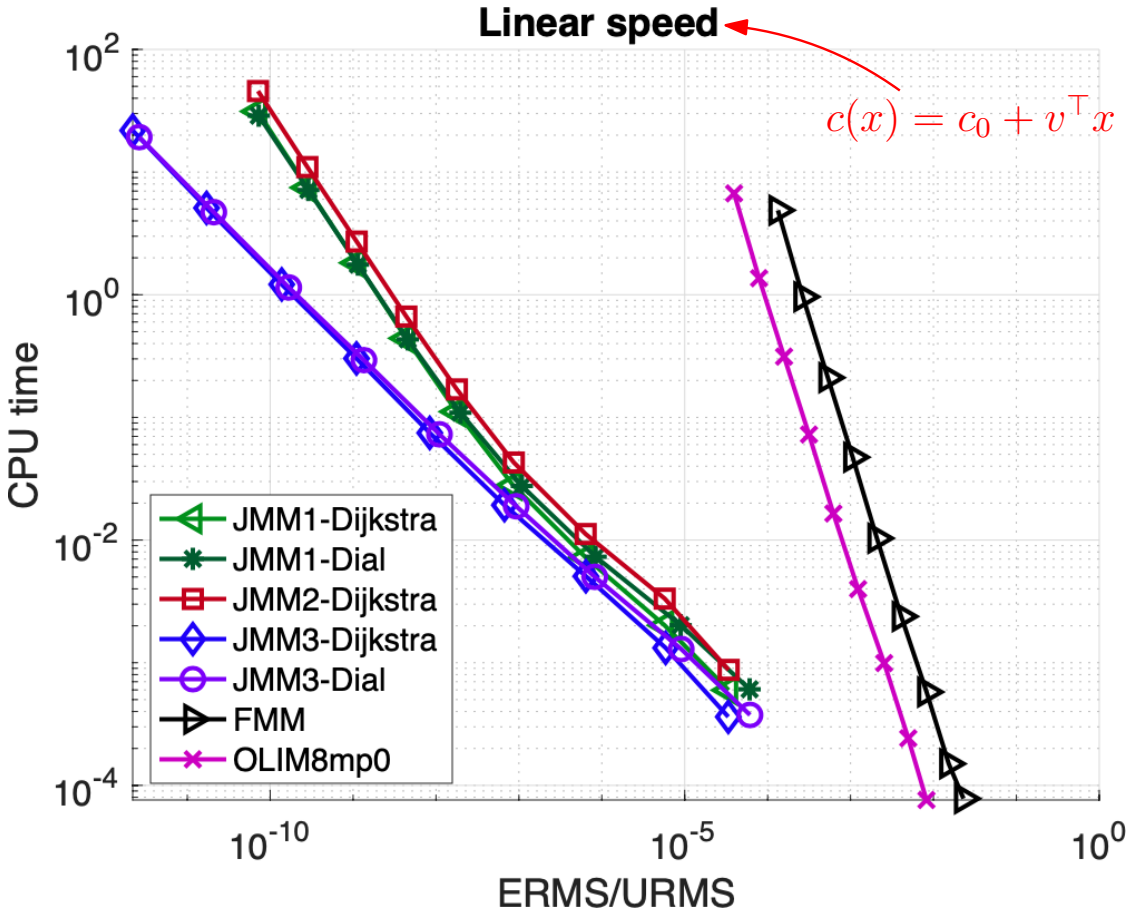
Afterwards, compute amplitude from:

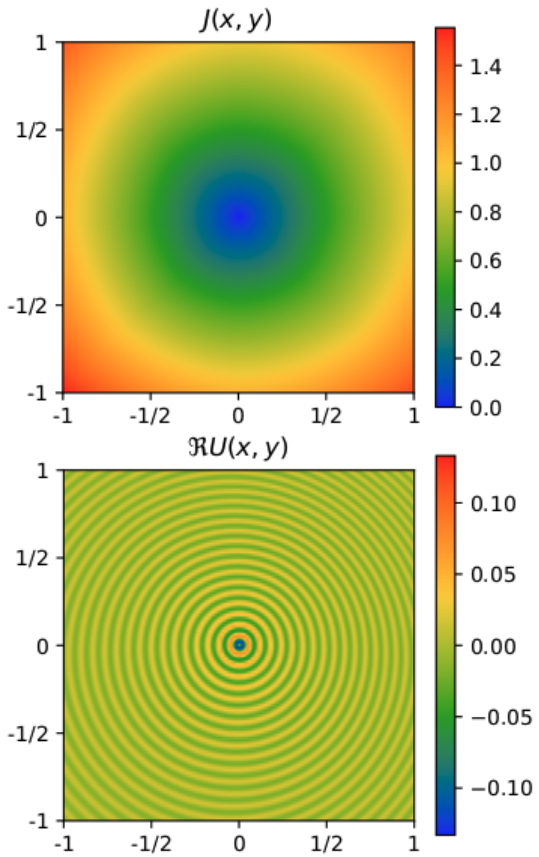
$$\alpha(\boldsymbol{x}) = \frac{e^{i\pi/4}}{2\sqrt{2\pi\omega}} \sqrt{\frac{c(\boldsymbol{x})}{J(\boldsymbol{x})}}.$$

This is specialized for the 2D case.

Potter and Cameron, 2021.

2D JMMs: numerical results

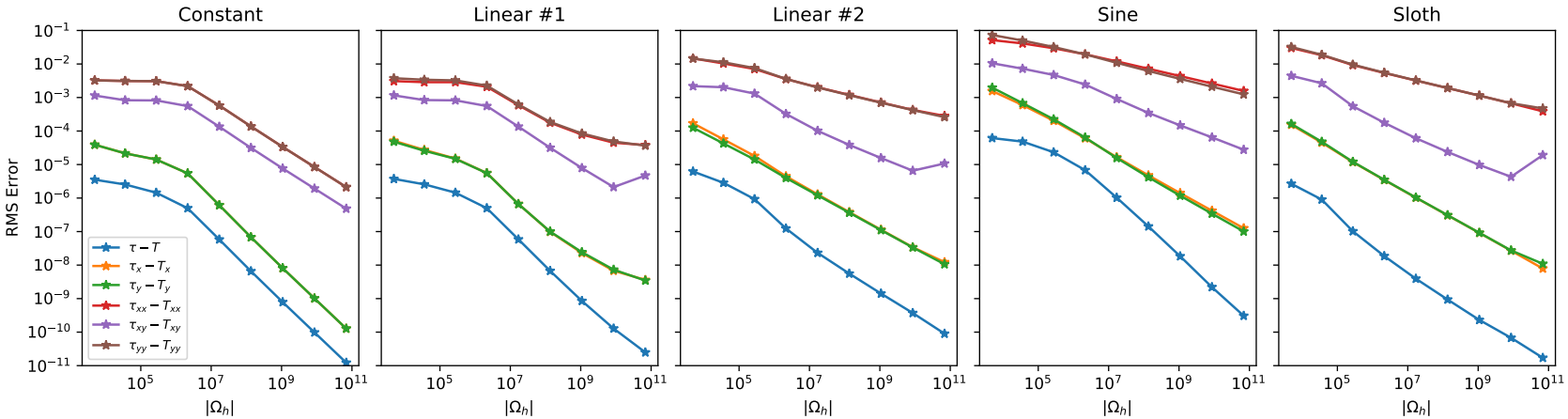




Potter and Cameron, 2021.

2D JMMs: numerical results

Results using cell marching:



Order of convergence ($O(h^p)$) for jets:

	$\tau - T$	$\tau_x - T_x$	$\tau_y - T_y$	$\tau_{xx} - T_{xx}$	$\tau_{xy} - T_{xy}$	$\tau_{yy} - T_{yy}$
Constant	3.09	3.11	3.11	2.01	2.05	2.01
Linear #1	2.99	2.43	2.40	1.39	2.01	1.39
Linear #2	2.10	1.76	1.72	0.77	1.25	0.77
Sine	2.91	1.80	1.89	0.73	1.31	0.80
Sloth	2.03	1.76	1.75	0.75	1.33	0.76

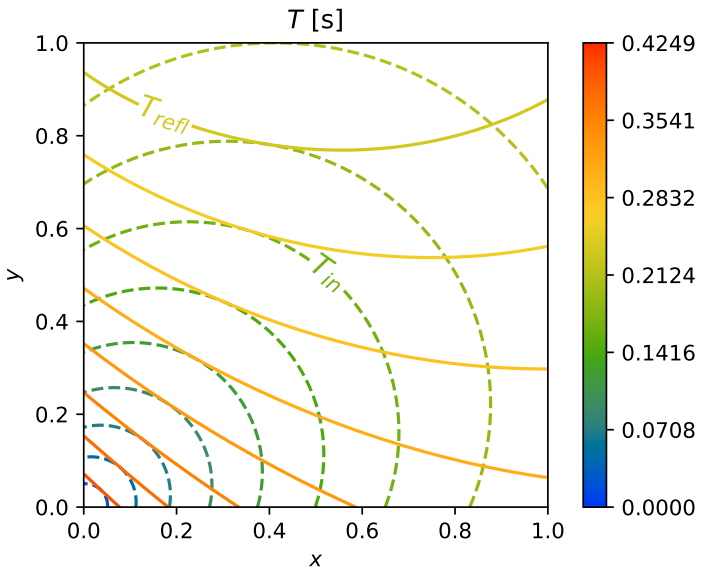
Potter and Cameron, 2021.

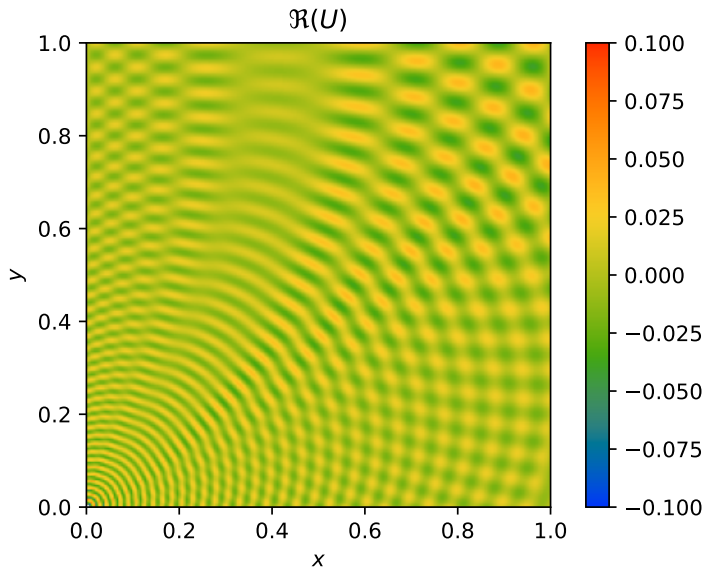
2D JMMs: reflections

Boundary conditions for reflected problem:

$$\tau_{\text{in}} = \tau_{\text{out}}, \quad J_{\text{in}} = J_{\text{out}} \quad \text{on wall}$$

and reflect $\nabla\tau_{\text{in}}$ over wall to get $\nabla\tau_{\text{out}}$:





2D JMMs: preliminary theoretical results

Theorem (JMM3 consistency)

Let $\Omega \subseteq \mathbb{R}^2$, assume that $s \in C^2(\Omega)$, and let $(\hat{\boldsymbol{x}}, \boldsymbol{x}_1, \boldsymbol{x}_2)$ be an update triangle where $\hat{\boldsymbol{x}}$ is `trial`*, and $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ are* `valid`*. Assume that no caustic is incident on the update triangle, and that the ray arriving at $\hat{\boldsymbol{x}}$ crosses the segment $[\boldsymbol{x}_1, \boldsymbol{x}_2]$. Then:*

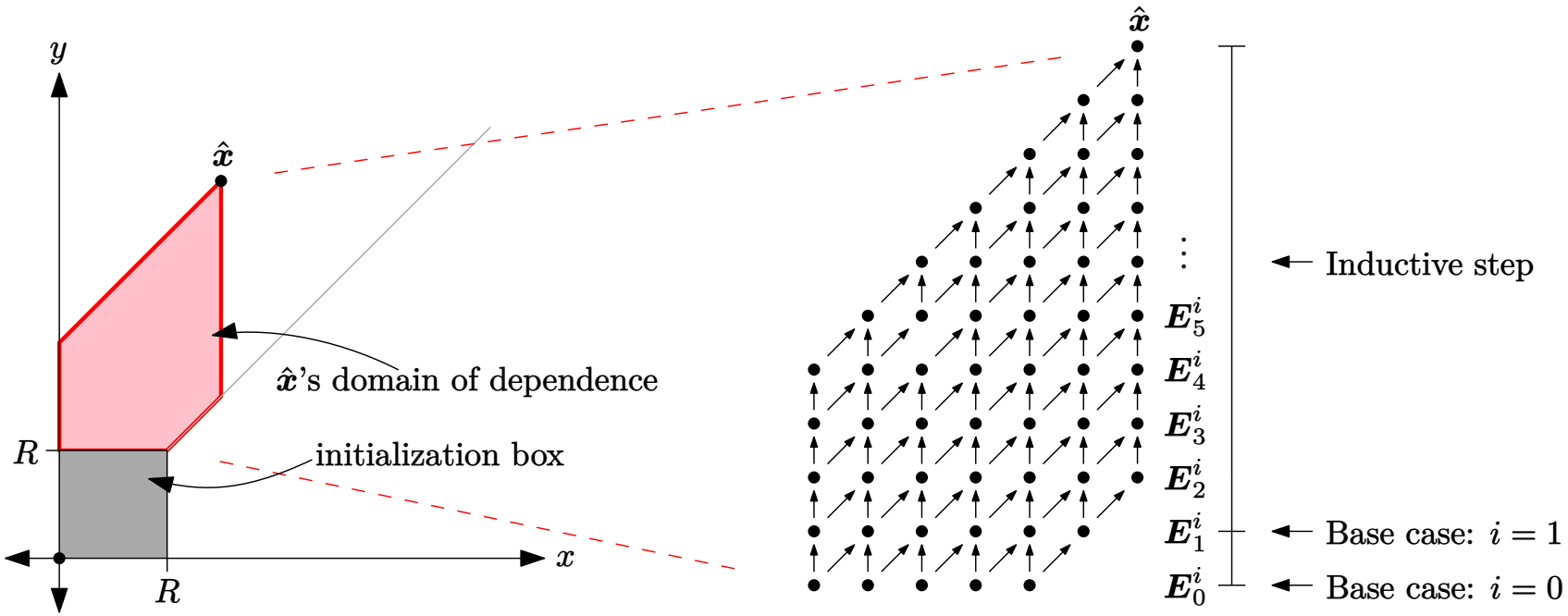
$$|\tau(\hat{\boldsymbol{x}}) - T(\hat{\boldsymbol{x}})| = O(h^4), \qquad \|\nabla\tau(\hat{\boldsymbol{x}}) - \nabla T(\hat{\boldsymbol{x}})\| = O(h^2). \tag{1}$$

Theorem (full accuracy for $c \equiv 1$)

For a point source at the origin with $c \equiv 1$, if T and ∇T are initialized in a region surrounding the origin with $O(1)$ diameter, then the 2D JMM computes a solution such that $|\tau(\boldsymbol{x}) - T(\boldsymbol{x})| = O(h^3)$ and $\|\nabla\tau(\boldsymbol{x}) - \nabla T(\boldsymbol{x})\| = O(h^3)$ for each node $\boldsymbol{x}$ of a regular grid

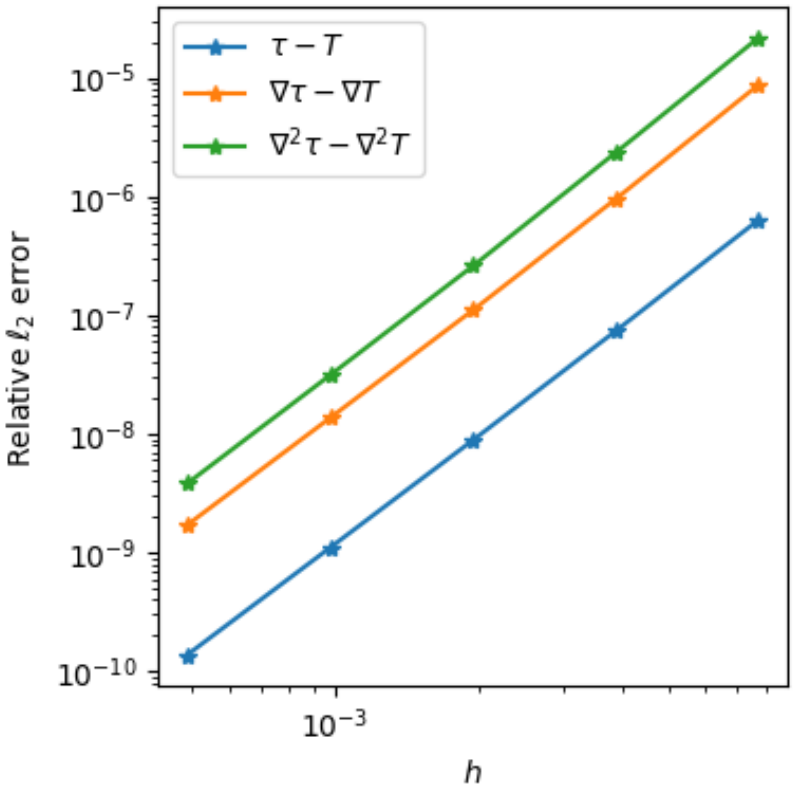
2D JMMs: error propagation ($c = 1$)

Follow the parents of each node back to the update box. Errors mix as we progress between layers of nodes. Combine this with consistency result to get overall bound on error.

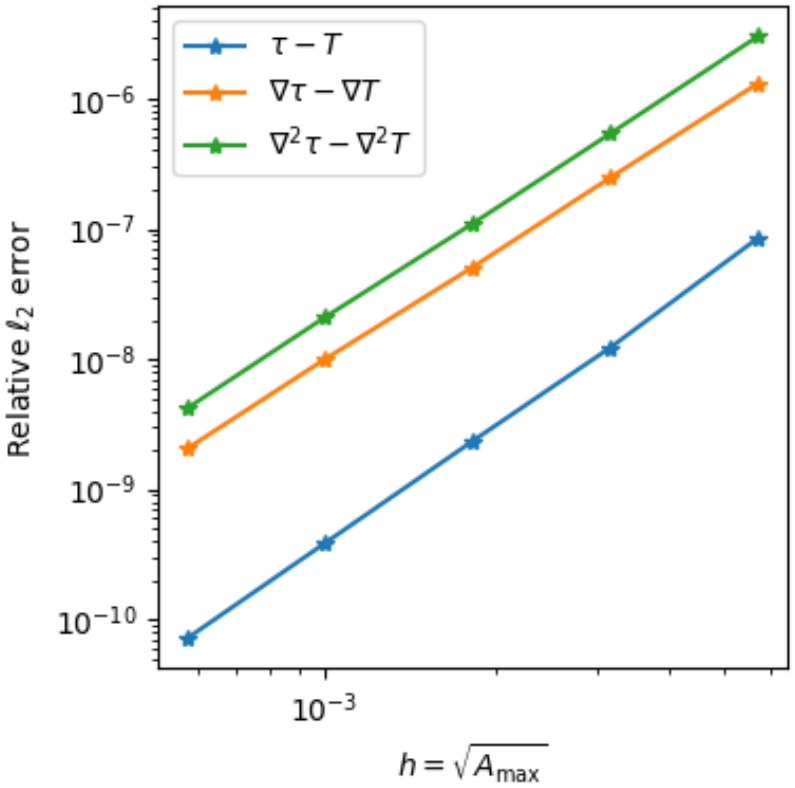


2D JMMs: global error ($c = 1$)

Regular grid JMM



Unstructured mesh JMM



	Regular		Unstructured	
Error	p	C	p	C
$\tau - T$	**3.051**	0.533	**3.064**	-0.513
$\nabla\tau - \nabla T$	**3.087**	3.302	**2.806**	0.936
$\nabla^2\tau - \nabla^2 T$	**3.128**	4.424	**2.854**	2.005

A JMM for an unstructured tetrahedron mesh

3D JMM: setup

Develop a JMM to solve the eikonal equation on a tetrahedron mesh in 3D

Assume that the speed of sound is constant (WLOG, $c \equiv 1$, so we compute the distance transform)

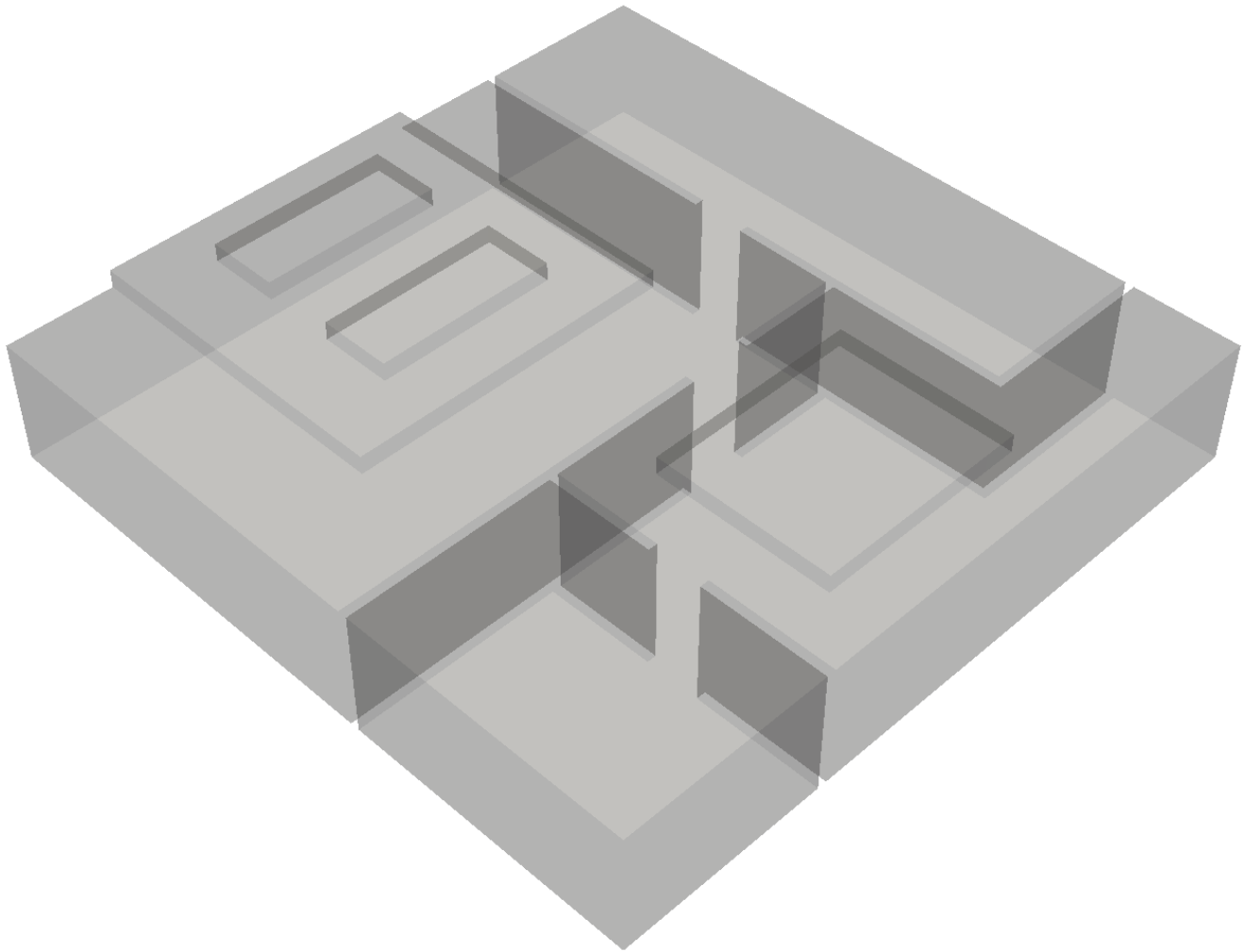
Goals: $O(N \log N)$ solver with at least $O(h^2)$ accurate T and ∇T, so that we can compute α consistently

Extend to spatially varying c later, since multiple arrival problem is already very complicated in 3D

Changes for varying c: have to solve slightly harder optimization problem for local updates, and have to deal with free-space caustics

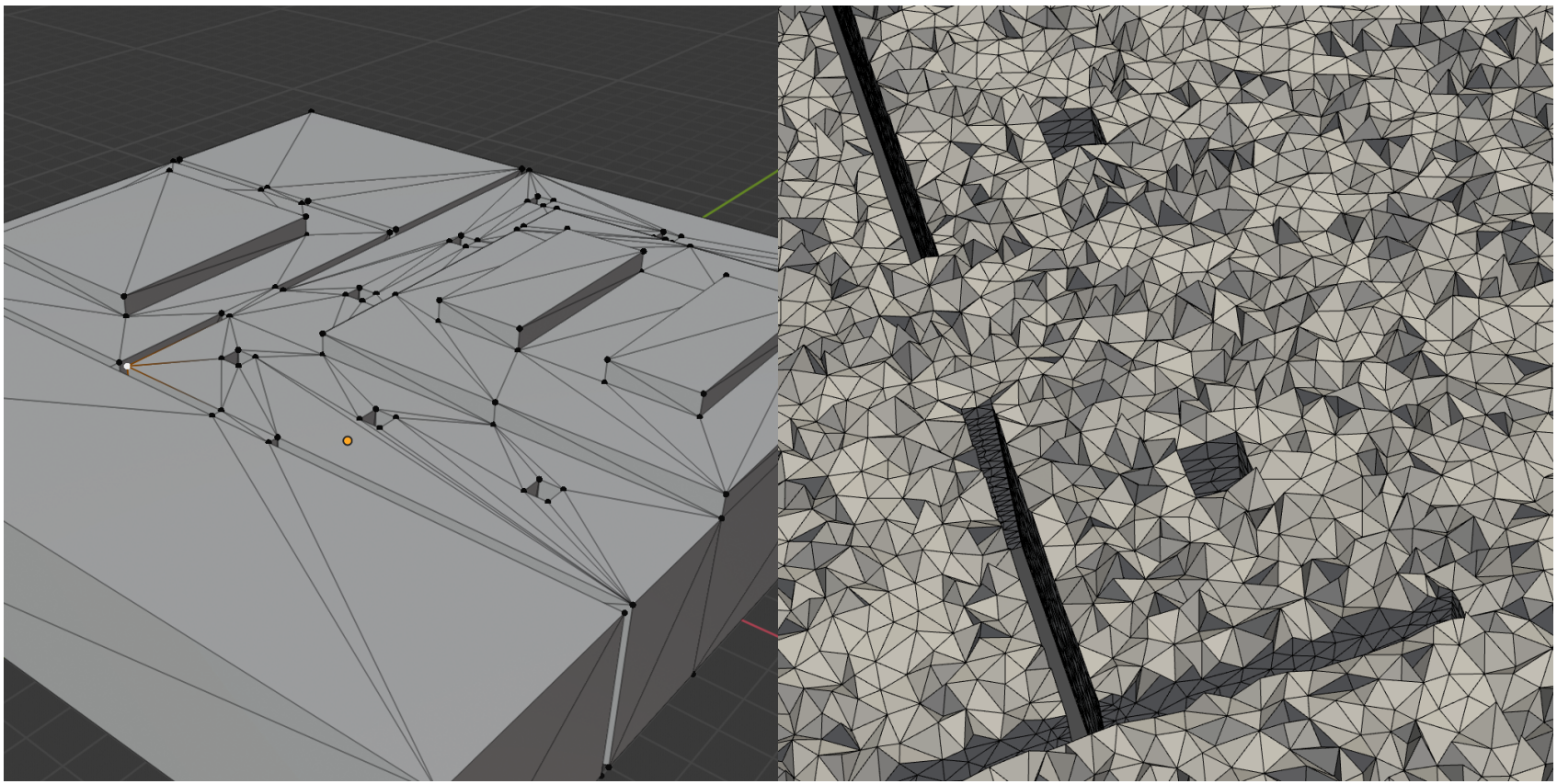
3D JMM: building test problem

Realistic test problem which includes some common architectural features, but not fully general (no small scale features, no curved surfaces, no floating obstacles, ...):



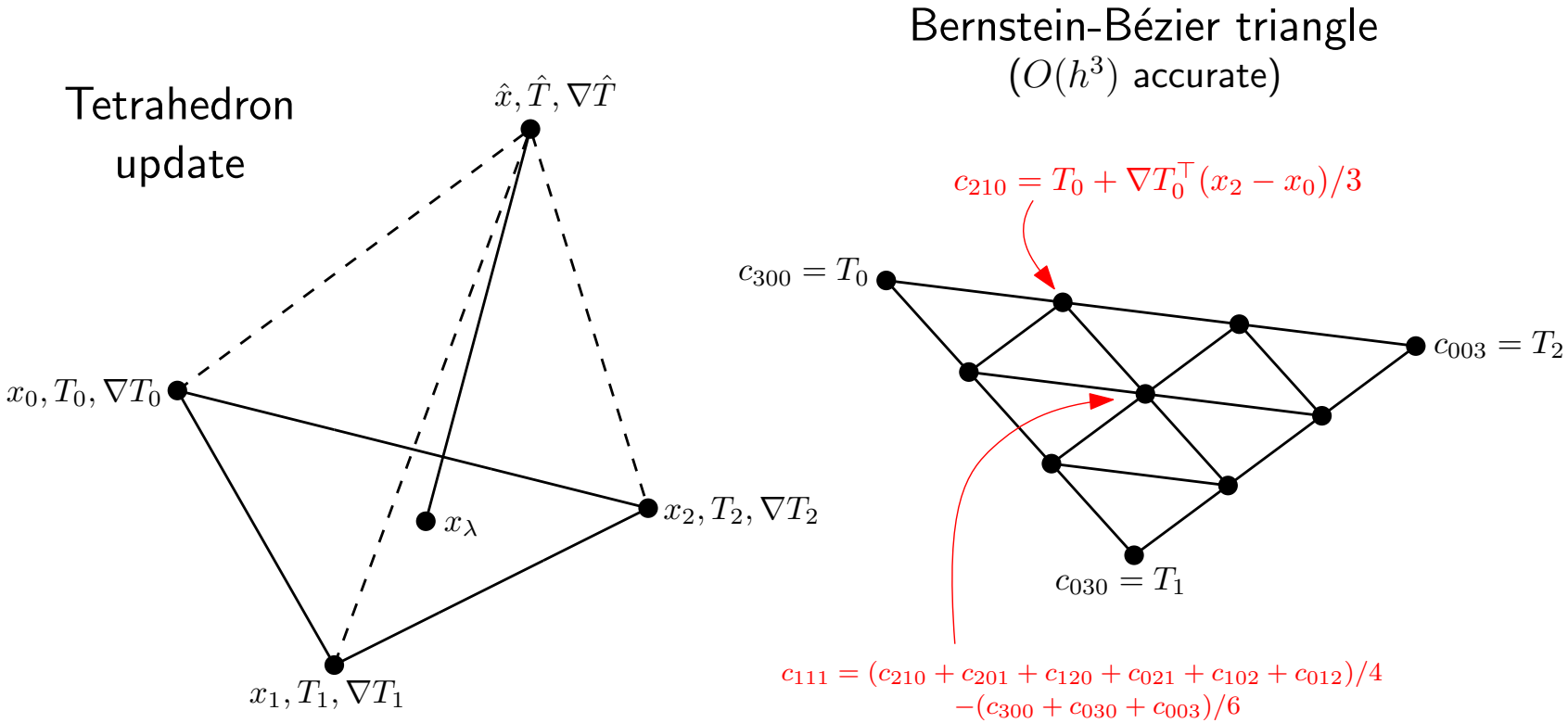
3D JMM: tetrahedron meshing

Modeled the test problem using Blender, discretized the boundary using tetgen:



3D JMM: tetrahedron updates

Workhorse for this solver is a cubically accurate tetrahedron update. Since $c \equiv 1$, rays are straight lines:

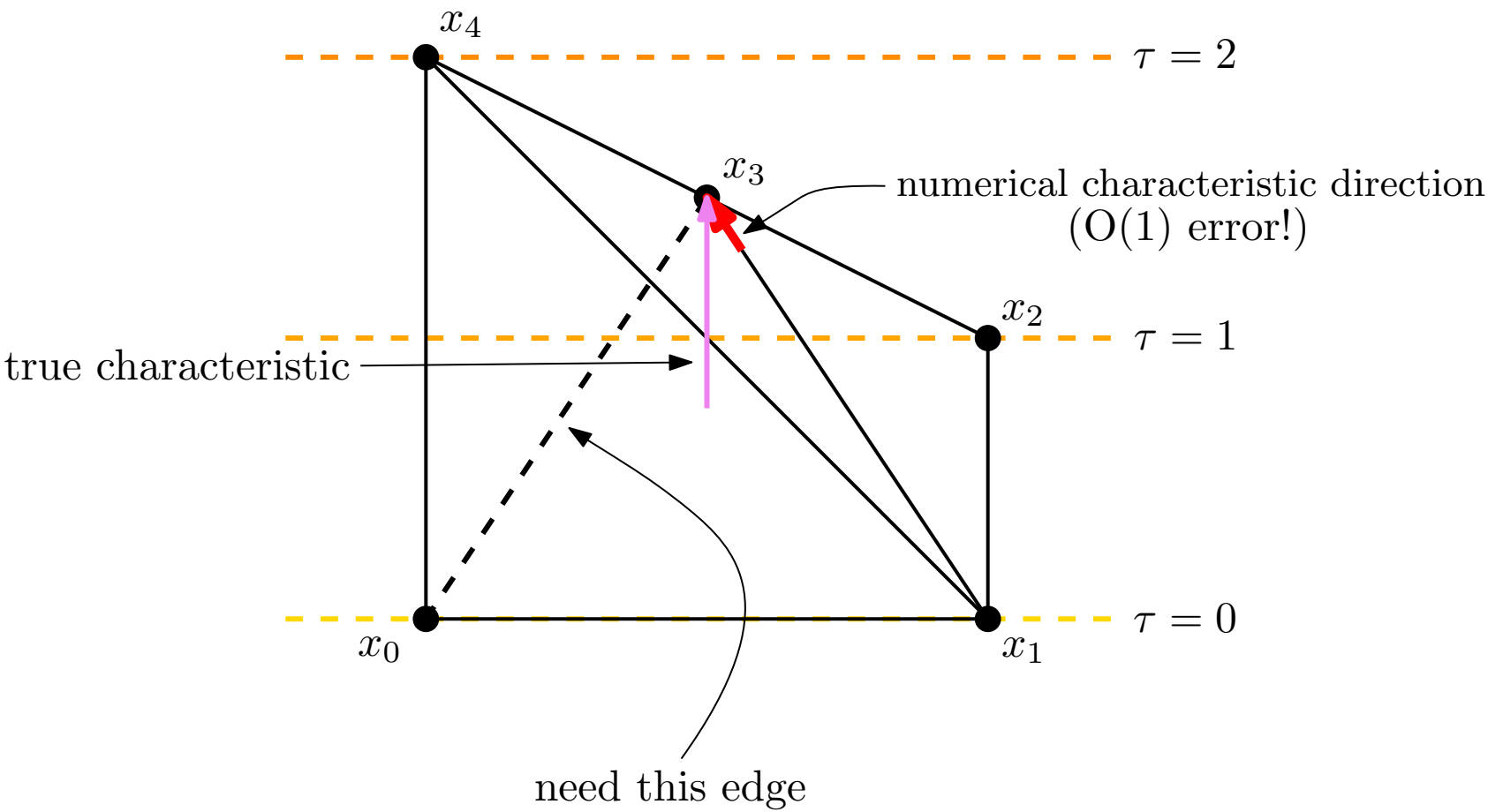


Minimize the cost function:

$$T(\hat{\boldsymbol{x}}) = \min_{\boldsymbol{\lambda}} \left\{ T(\boldsymbol{x}_{\boldsymbol{\lambda}}) + \|\hat{\boldsymbol{x}} + \boldsymbol{x}_{\boldsymbol{\lambda}}\| \right\}$$

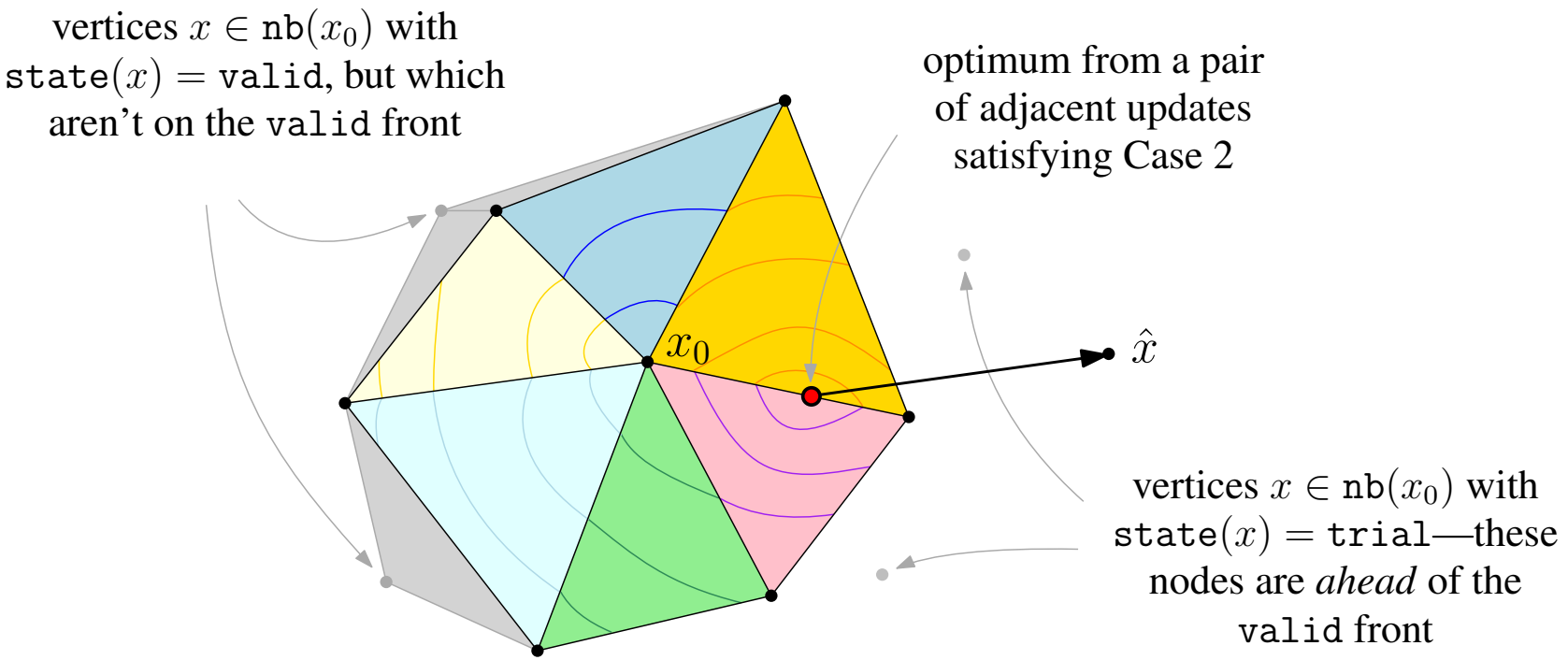
3D JMM: causality

We can't use the topology of the tetrahedron mesh to define update neighborhoods:



3D JMM: enforcing causality

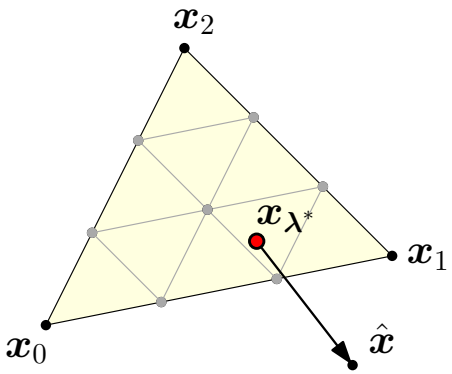
We want to find the point $\boldsymbol{x}_{\boldsymbol{\lambda}^*}$ which minimizes the cost function over each `valid` triangle on the `valid` front



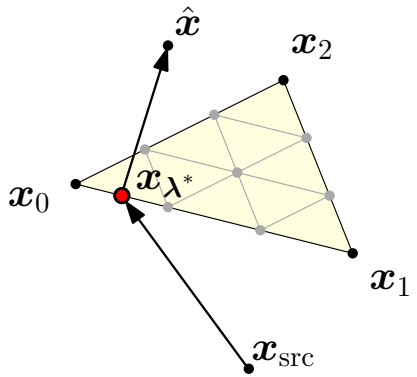
3D JMM: enforcing causality

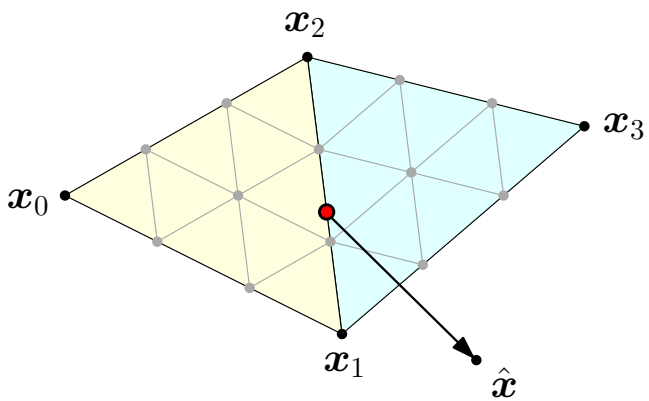
Check the Lagrange multipliers of a minimizer to see if the update is causal:

Interior point minimizer

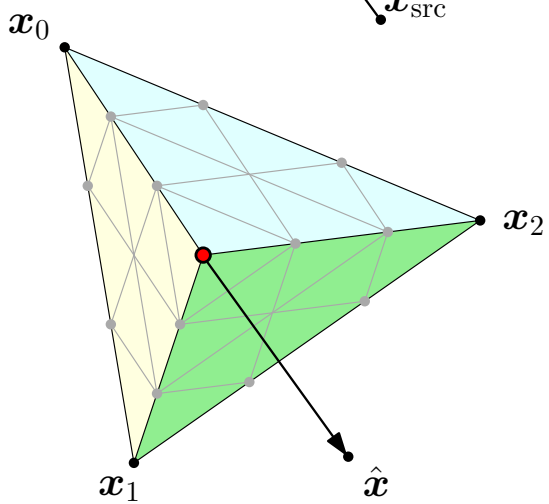


Unphysical ray



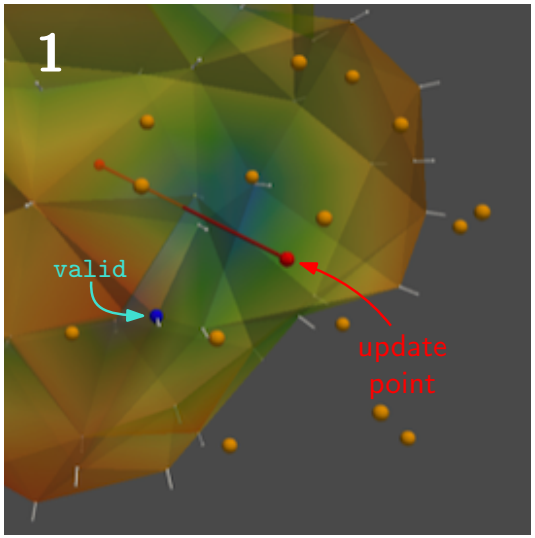


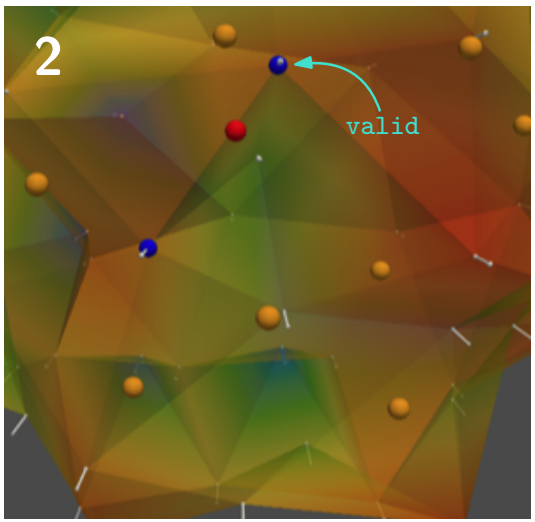
Boundary minimizer from two updates

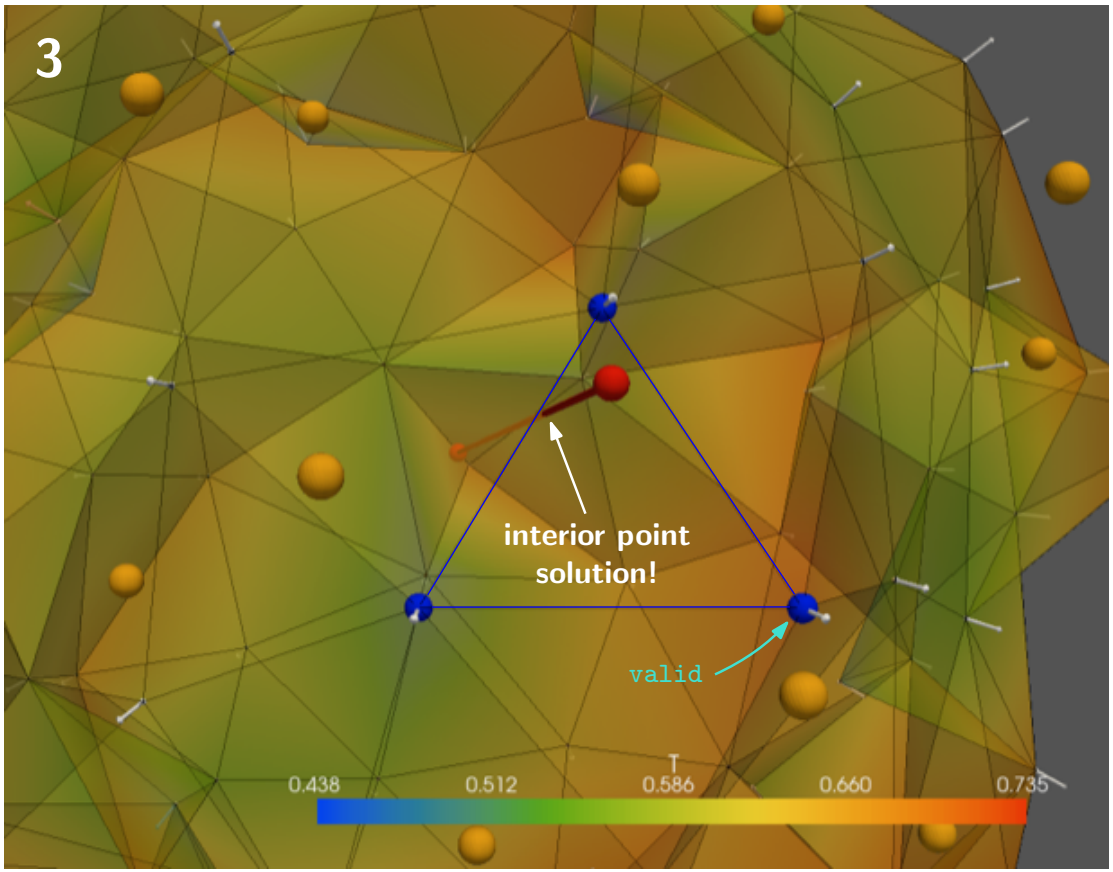


. . . and from three updates

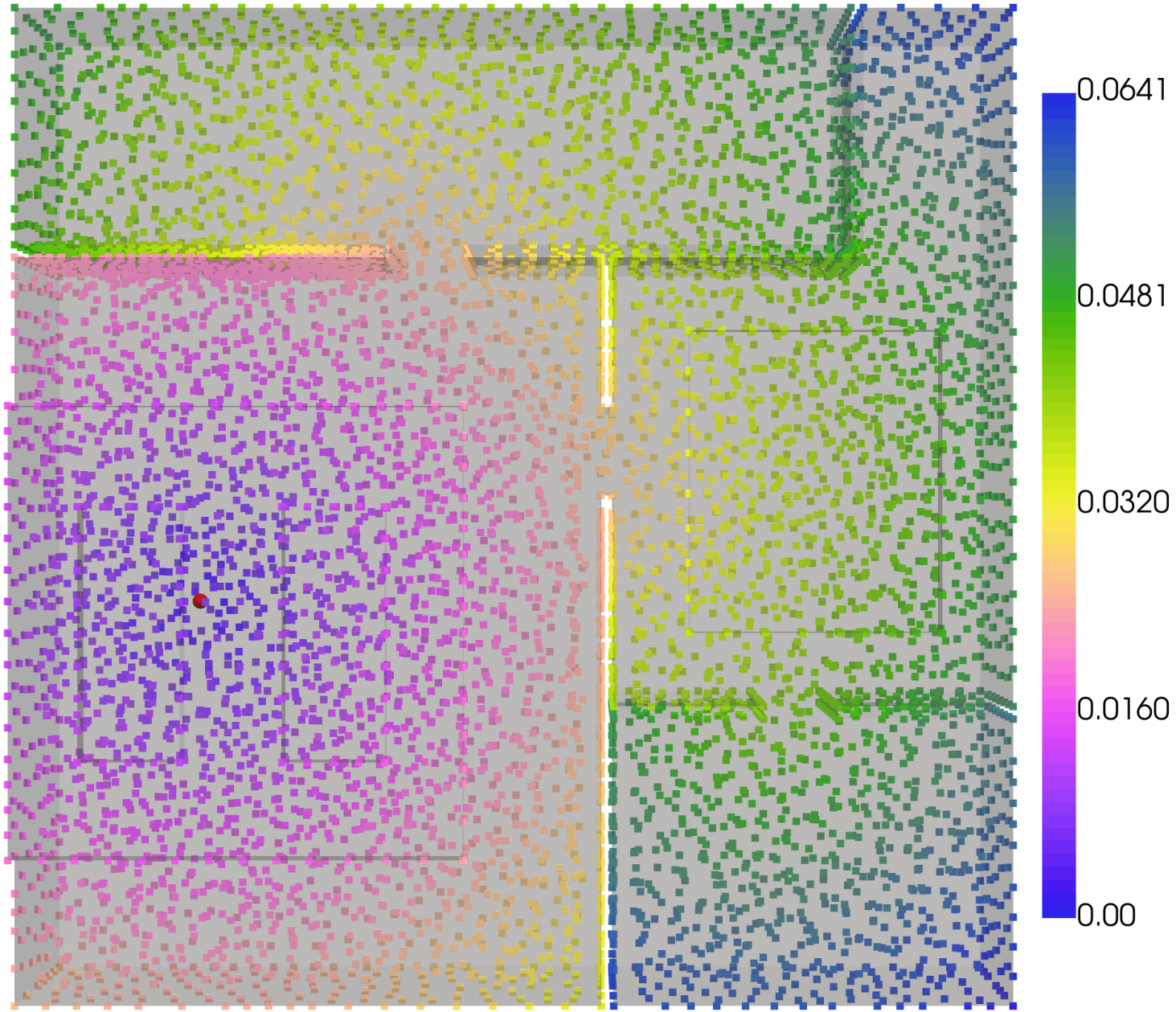
3D JMM: what does this look like?



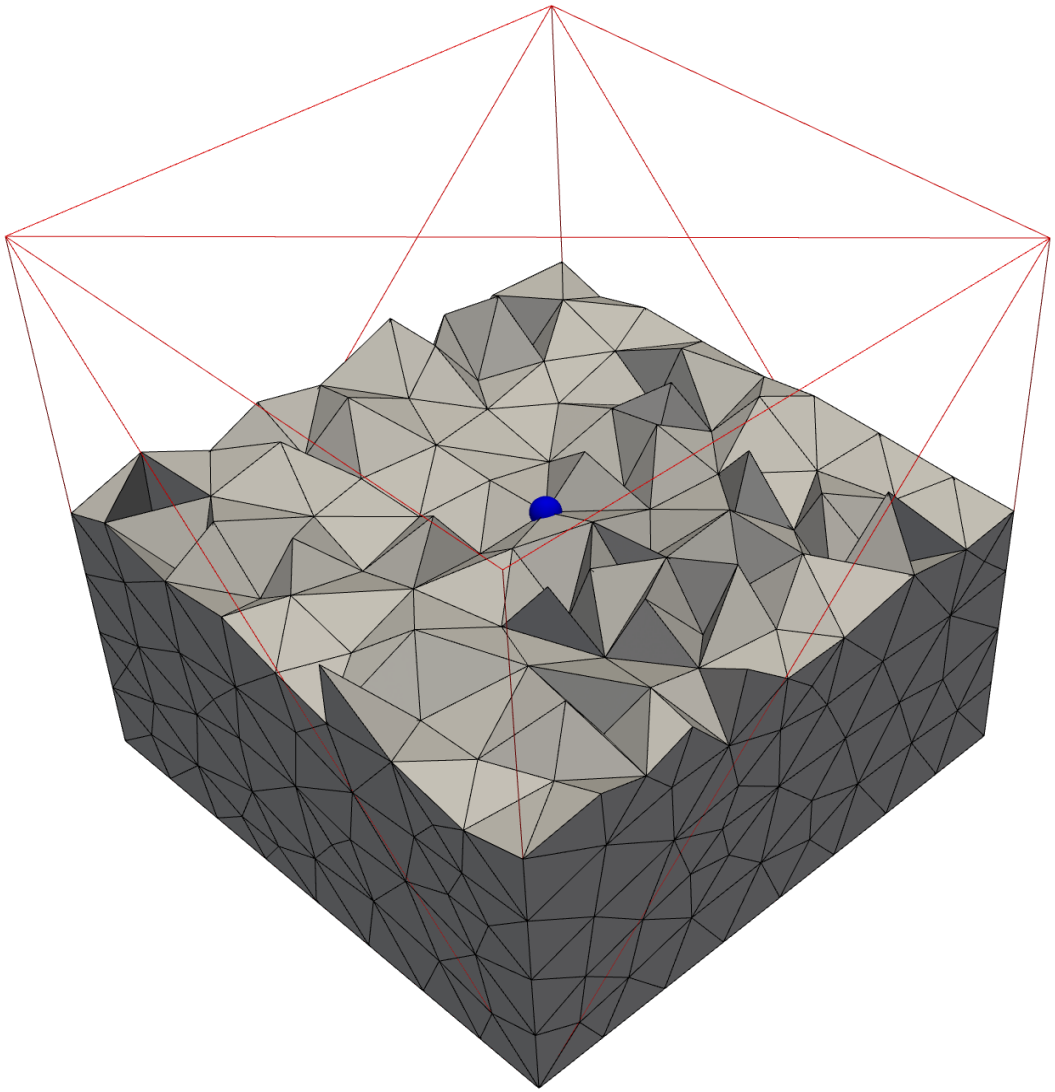




3D JMM: eikonal on the test building domain



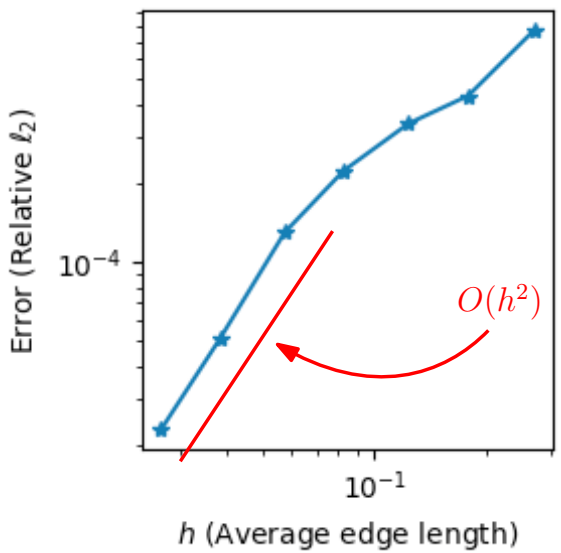
3D JMM: testing on a box domain



▲Input mesh

▲Tetrahedron mesh

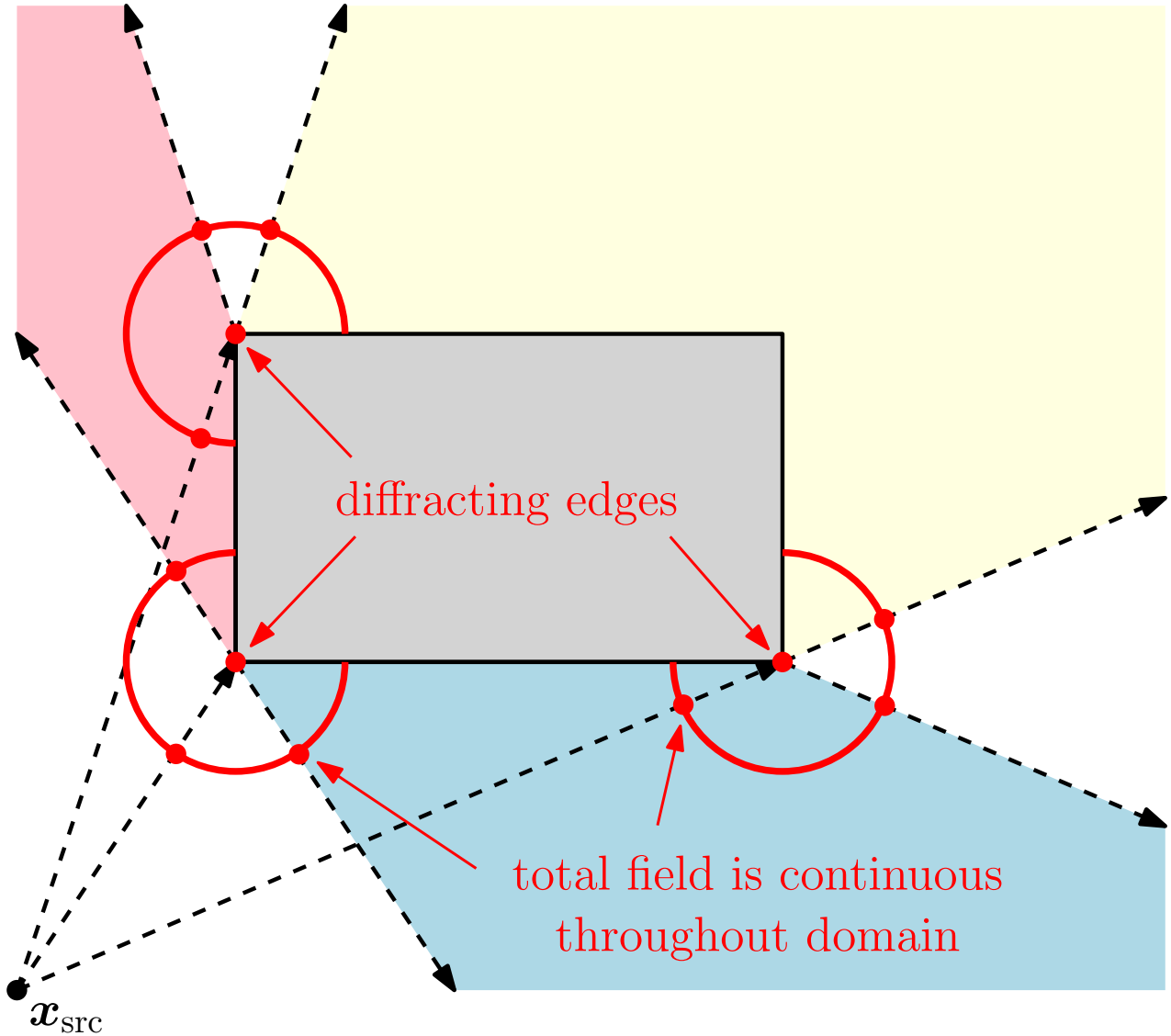
▲Point source



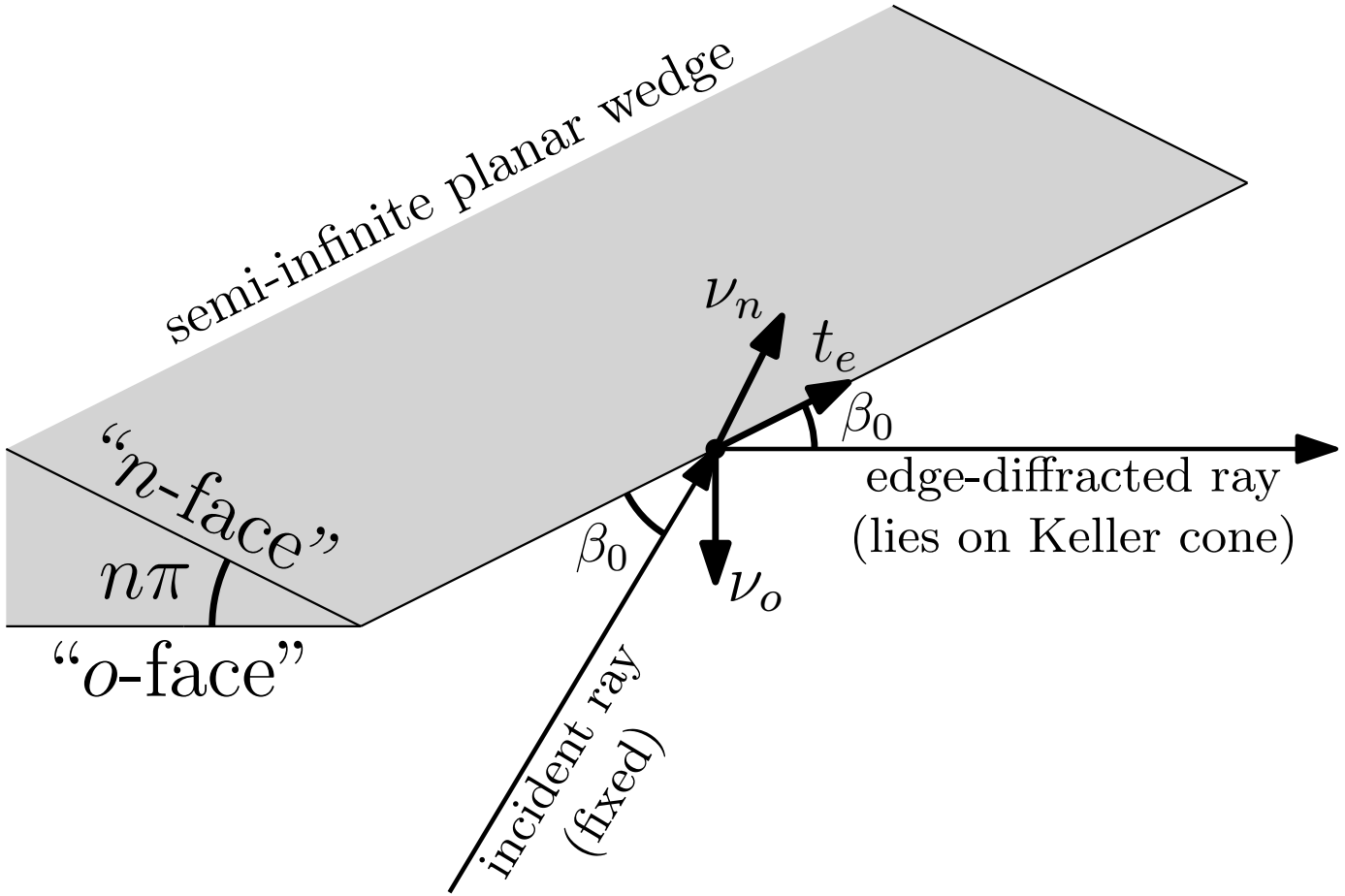
Multiple arrivals in 3D

MA in 3D: multiple arrivals

Multiple arrivals: $u_\omega(\boldsymbol{x}) \sim \sum_{\substack{k\text{th ray} \\ \text{through } \boldsymbol{x}}} A_\omega^{(k)}(\boldsymbol{x}) e^{i\omega\tau^{(k)}(\boldsymbol{x})}$



MA in 3D: edge diffraction



Generalized Fermat's principle with boundary constraints:

$$\min_{\boldsymbol{\psi}} \int_{\boldsymbol{\psi}} s(\boldsymbol{\psi}) d\ell \quad \text{s.t.} \quad \boldsymbol{\psi}_i \in \partial\Omega_i$$

MA in 3D: BCs for reflected and diffracted eikonal

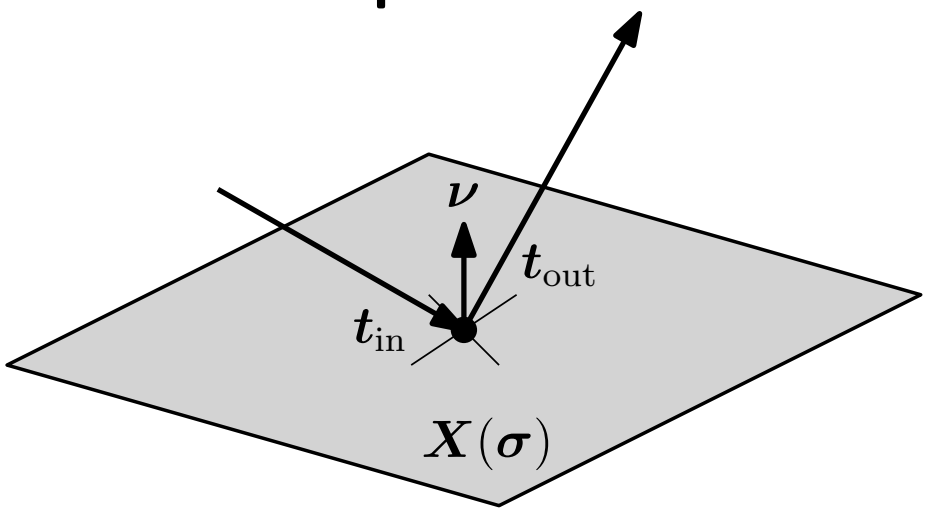
Boundary conditions for the reflected and diffracted eikonal:

$$\tau_{\text{in}}(\boldsymbol{X}(\boldsymbol{\sigma})) = \tau_{\text{out}}(\boldsymbol{X}(\boldsymbol{\sigma}))$$

Taking the gradient gives:

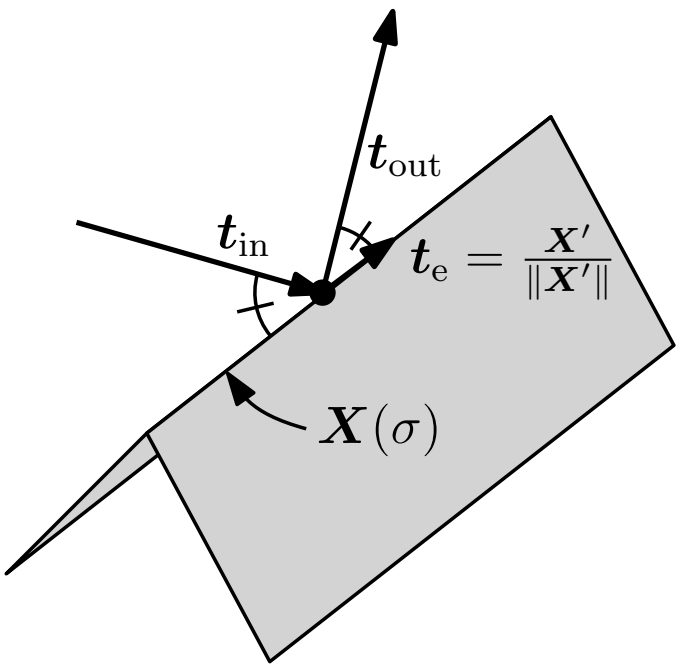
$$\nabla \boldsymbol{X}(\boldsymbol{\sigma})^\top \nabla \tau_{\text{in}}(\boldsymbol{X}(\boldsymbol{\sigma})) = \nabla \boldsymbol{X}(\boldsymbol{\sigma})^\top \nabla \tau_{\text{out}}(\boldsymbol{X}(\boldsymbol{\sigma}))$$

This leads to **specular reflection** and the **Keller cone**:



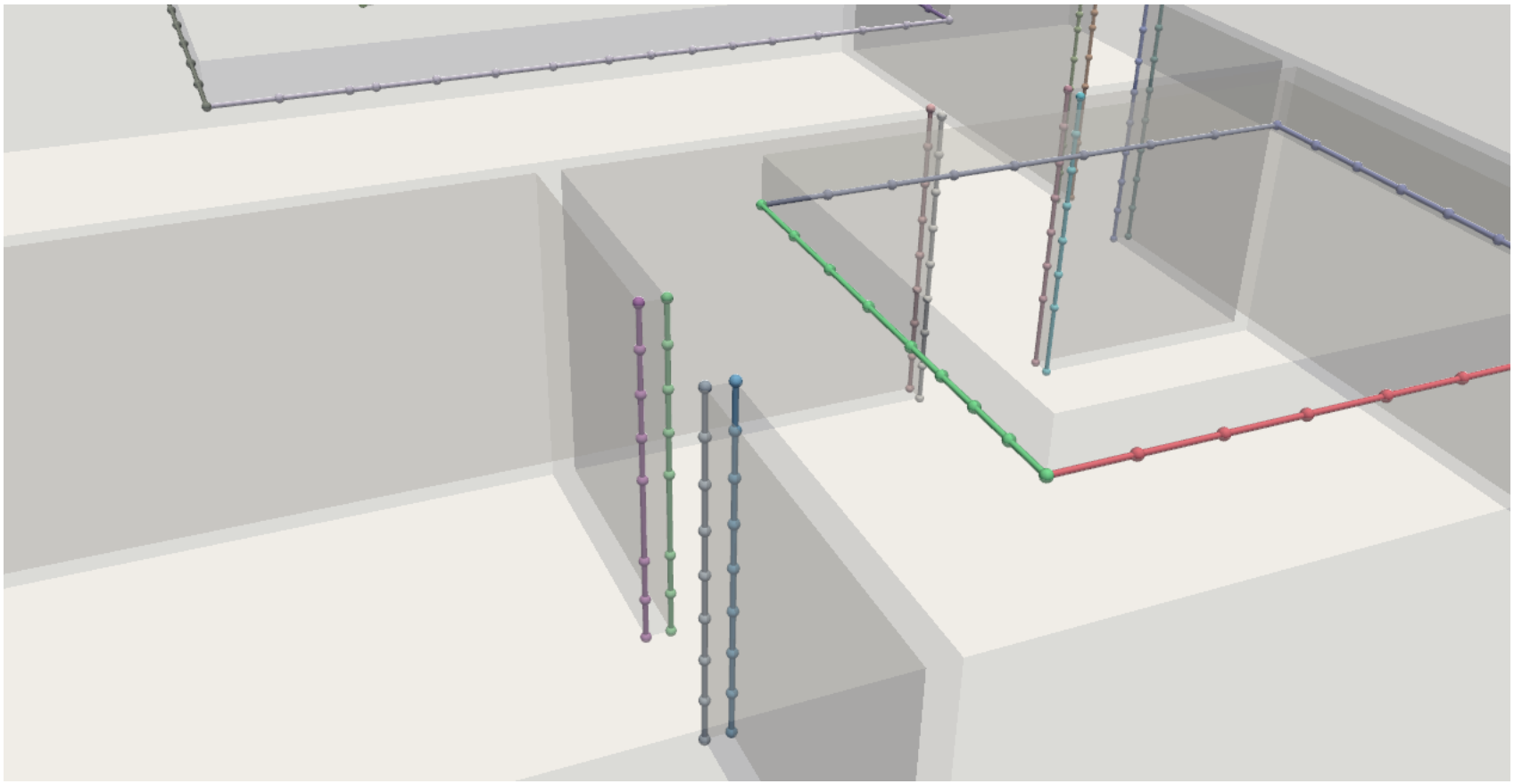
$$\boldsymbol{\nu} \in \text{null}(\boldsymbol{X})$$

$$\boldsymbol{t}_{\text{out}} = \left(\boldsymbol{I} - 2\boldsymbol{\nu}\boldsymbol{\nu}^\top\right)\boldsymbol{t}_{\text{in}}$$



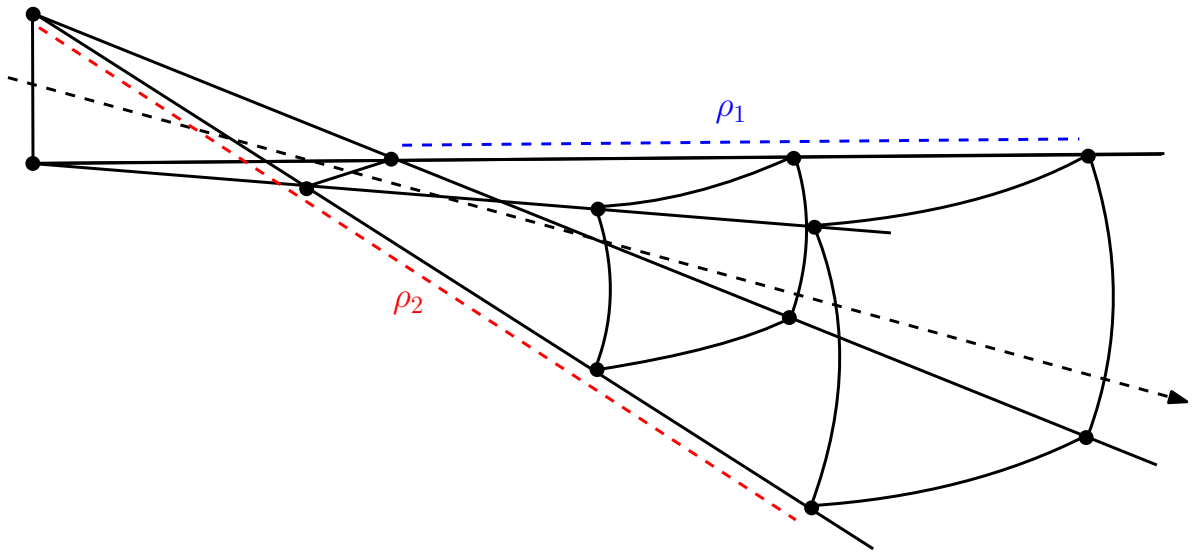
MA in 3D: reflectors and diffractors

Restrict τ_{in} to reflecting and diffracting subsets of $\partial\Omega$ and compute scattered fields:



MA in 3D: the reflected amplitude

Astigmatic pencil of rays:



The amplitude for a reflection is:

$$\alpha_{\text{out}}(\boldsymbol{x}) = R(\boldsymbol{x}_0)\sqrt{\frac{\rho_1(\boldsymbol{x}_0)\rho_2(\boldsymbol{x}_0)}{\left[\rho_1(\boldsymbol{x}_0)+\sigma(\boldsymbol{x},\boldsymbol{x}_0)\right]\left[\rho_2(\boldsymbol{x}_0)+\sigma(\boldsymbol{x};\boldsymbol{x}_0)\right]}}\,\alpha_{\text{in}}(\boldsymbol{x}_0),$$

McNamara, 1990.

MA in 3D: the diffracted amplitude

Amplitude for edge-diffracted ray is:

$$\alpha_{\text{out}}(\boldsymbol{x}) = D(\boldsymbol{x}_0)\sqrt{\frac{\rho_e(\boldsymbol{x}_0)}{\sigma(\boldsymbol{x};\boldsymbol{x}_0)\left[\rho_e(\boldsymbol{x}_0)+\sigma(\boldsymbol{x};\boldsymbol{x}_0)\right]}}\,\alpha_{\text{in}}(\boldsymbol{x}_0).$$

Complexity hidden in the UTD coefficient $D(\boldsymbol{x}_0)$:

$$D = D_1 + D_2 + R\cdot(D_3 + D_4),$$

$$D_1 = \frac{-e^{-i\pi/4}}{2n\sqrt{2\pi k}\sin\beta_0}\cot\left(\frac{\pi+\beta^-}{2n}\right)F\left(kL^{\text{in}}a^+(\beta^-)\right),$$

$$D_2 = \frac{-e^{-i\pi/4}}{2n\sqrt{2\pi k}\sin\beta_0}\cot\left(\frac{\pi-\beta^-}{2n}\right)F\left(kL^{\text{in}}a^-(\beta^-)\right),$$

$$D_3 = \frac{-e^{-i\pi/4}}{2n\sqrt{2\pi k}\sin\beta_0}\cot\left(\frac{\pi+\beta^+}{2n}\right)F\left(kL^{\text{in}}a^+(\beta^+)\right),$$

$$D_4 = \frac{-e^{-i\pi/4}}{2n\sqrt{2\pi k}\sin\beta_0}\cot\left(\frac{\pi-\beta^+}{2n}\right)F\left(kL^{\text{in}}a^-(\beta^+)\right).$$

MA in 3D: marching the Hessian

The wavefront curvature in section spanned by $\boldsymbol{v}$ and ∇T is:

$$\kappa_{\boldsymbol{v}}(\boldsymbol{x}) = -\frac{\boldsymbol{v}^\top \nabla^2 T(\boldsymbol{x}) \boldsymbol{v}}{s(\boldsymbol{x})}, \qquad \rho_{\boldsymbol{v}}(\boldsymbol{x}) = \frac{1}{\kappa_{\boldsymbol{v}}(\boldsymbol{x})}$$

The Hessian near the point source for $c \equiv 1$ is:

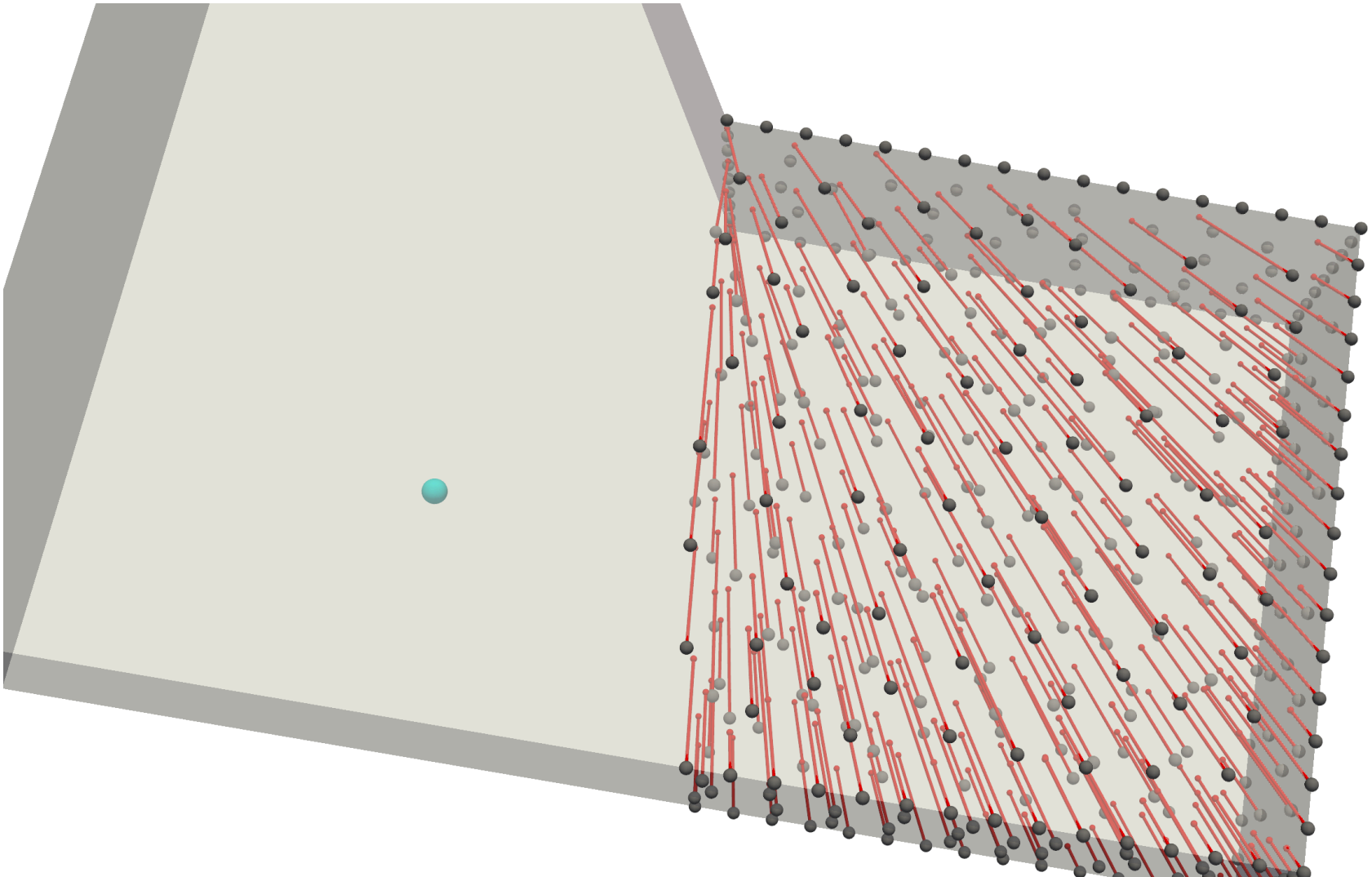
$$\nabla^2 T(\boldsymbol{x}) = \frac{1}{T(\boldsymbol{x})} \Big(\boldsymbol{I} - \nabla T(\boldsymbol{x}) \otimes \nabla T(\boldsymbol{x}) \Big).$$

Propagate the Hessian along ψ using paraxial RT:

$$\nabla^2 T(\hat{\boldsymbol{x}}) = \nabla^2 T(\boldsymbol{x}_{\boldsymbol{\lambda}}) \Big(\boldsymbol{I} + \|\hat{\boldsymbol{x}} - \boldsymbol{x}_{\boldsymbol{\lambda}}\| \nabla^2 T(\boldsymbol{x}_{\boldsymbol{\lambda}}) \Big)^{-1}.$$

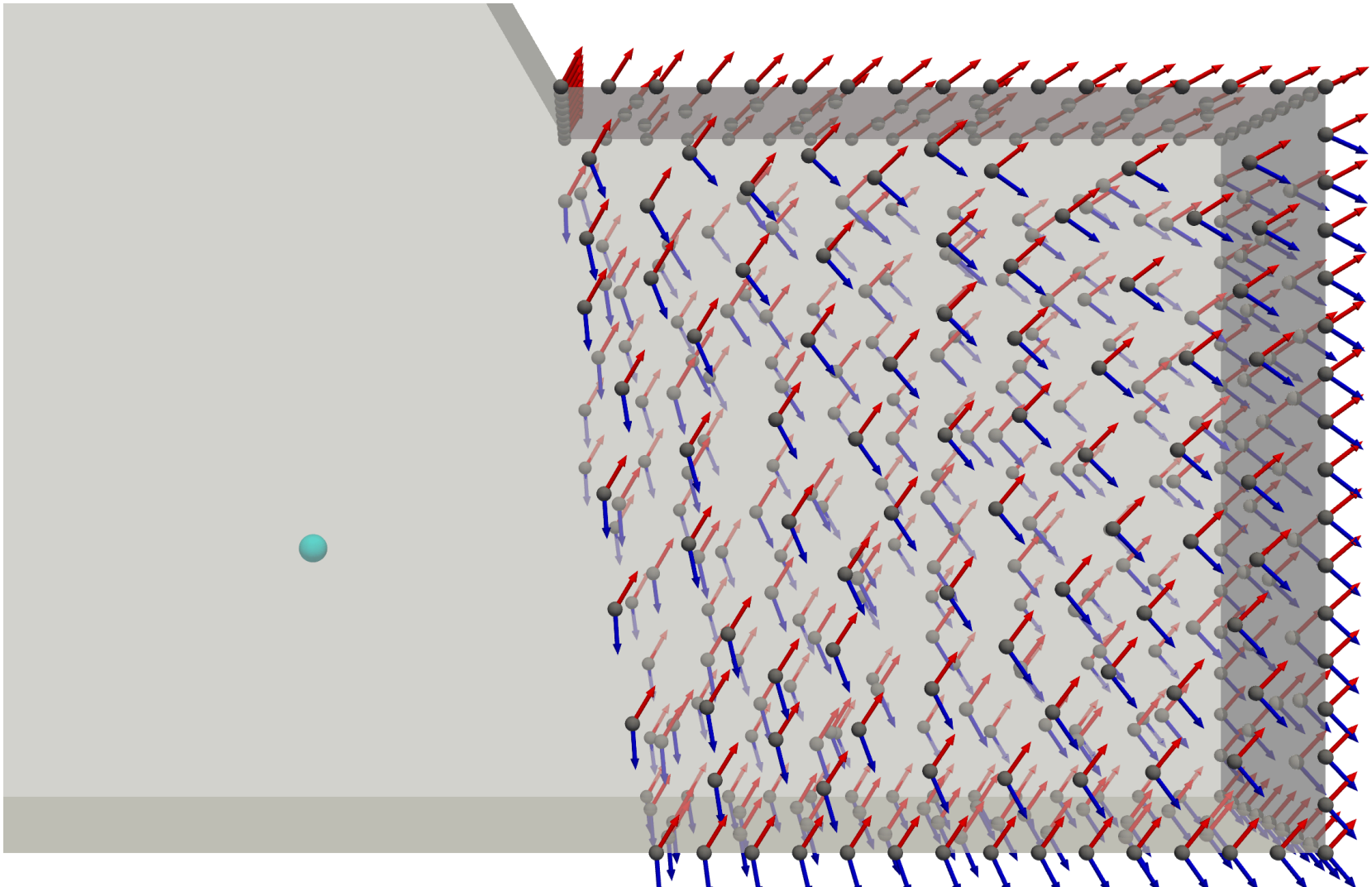
MA in 3D: evaluating BCs using dynamic programming

For $\boldsymbol{x} \in \Omega$, need to evaluate $f(\boldsymbol{x}_0)$, where $\boldsymbol{x}_0 \in \partial\Omega$ and a ray leads from $\boldsymbol{x}_0$ to $\boldsymbol{x}$. Do this after solving the eikonal equation using dynamic programming:



MA in 3D: the $\boldsymbol{t}_{\text{in}}$ and $\boldsymbol{t}_{\text{out}}$ fields

Two important vector fields are the tangent vector of the ray arriving at $\boldsymbol{x}_0$ and the ray leaving $\boldsymbol{x}_0$, which we call $\boldsymbol{t}_{\text{in}}(\boldsymbol{x})$ and $\boldsymbol{t}_{\text{out}}(\boldsymbol{x})$, respectively:



MA in 3D: shadow zones and tapering

We use $\boldsymbol{t}_{\text{in}}$ and $\boldsymbol{t}_{\text{out}}$ to taper the amplitude of points in shadow:

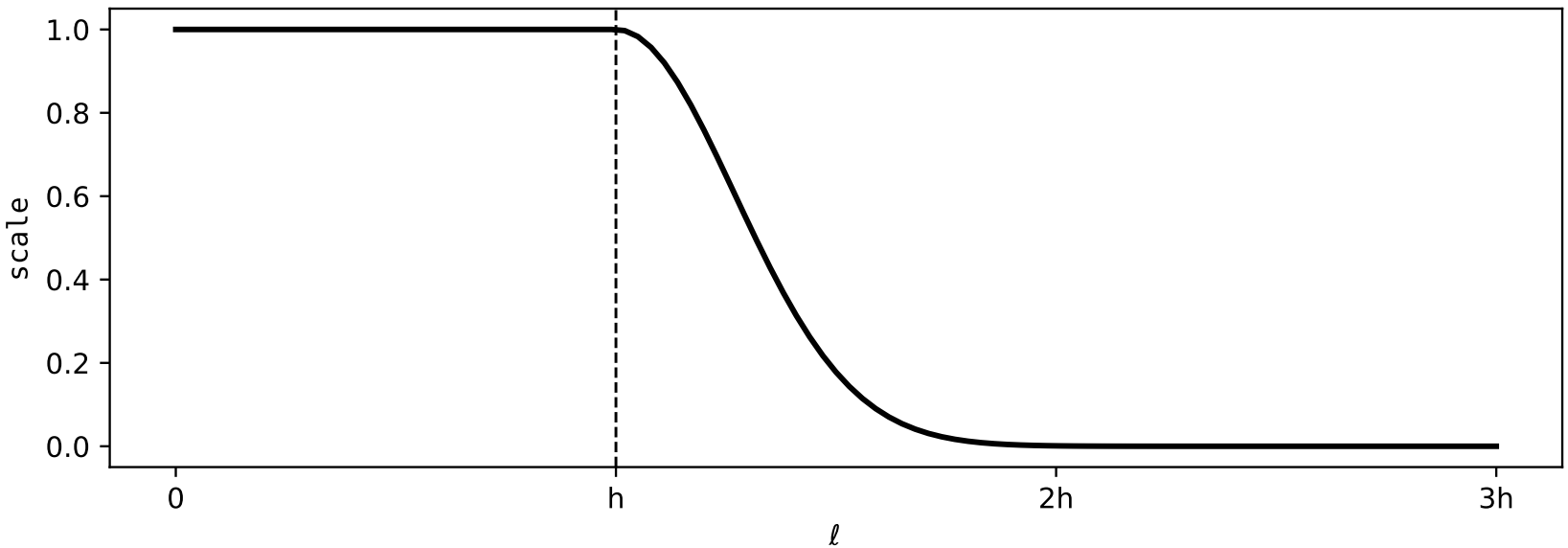
$$\ell_Z(\boldsymbol{x}) = \cos^{-1}(\boldsymbol{t}_{\text{out}}(\boldsymbol{x}) \cdot \nabla T(\boldsymbol{x}))$$

and points violating reflection and diffraction angle conditions:

$$\ell_R(\boldsymbol{x}) = \cos^{-1}\Big(\boldsymbol{t}_{\text{out}}(\boldsymbol{x})\big(\boldsymbol{I} - 2\boldsymbol{\nu}\boldsymbol{\nu}^\top\big)\boldsymbol{t}_{\text{in}}(\boldsymbol{x})\Big),$$

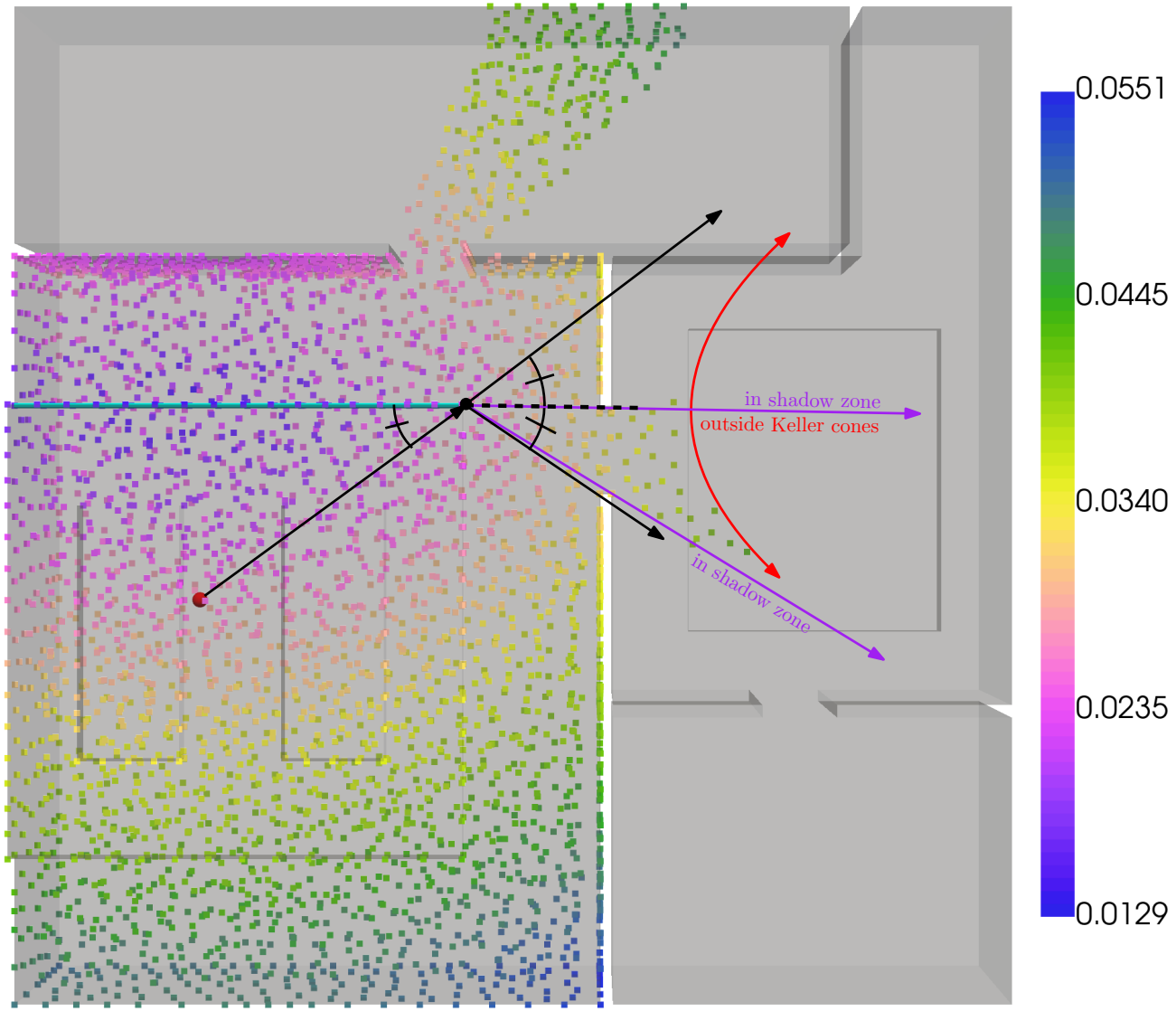
$$\ell_D(\boldsymbol{x}) = \Big|\cos^{-1}\big(\boldsymbol{t}_{\text{in}}(\boldsymbol{x}) \cdot \boldsymbol{t}_{\text{e}}\big) - \cos^{-1}\big(\boldsymbol{t}_{\text{out}}(\boldsymbol{x}) \cdot \boldsymbol{t}_{\text{e}}\big)\Big|$$

define `scale`(ℓ) to be Gaussian window above $\ell = O(h)$:

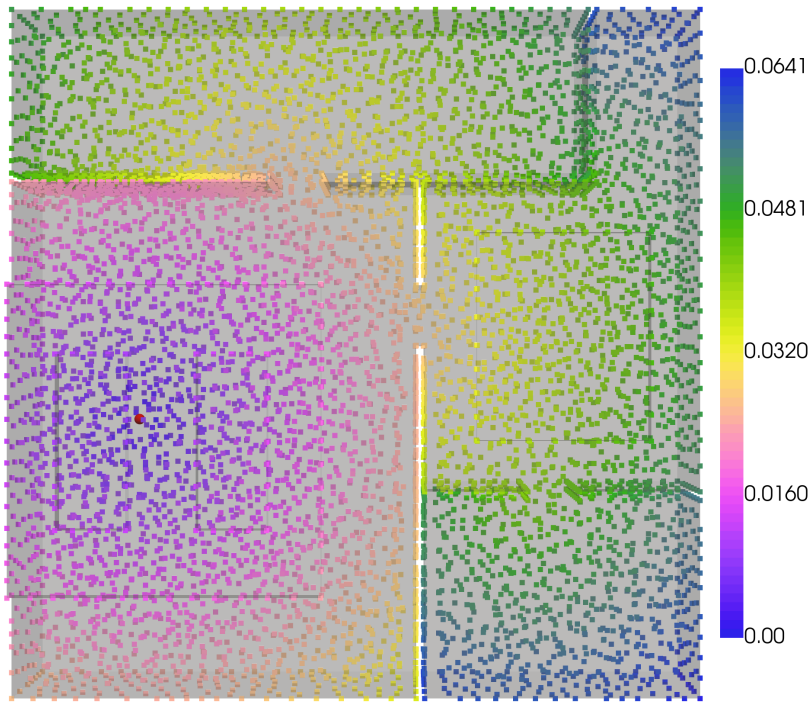


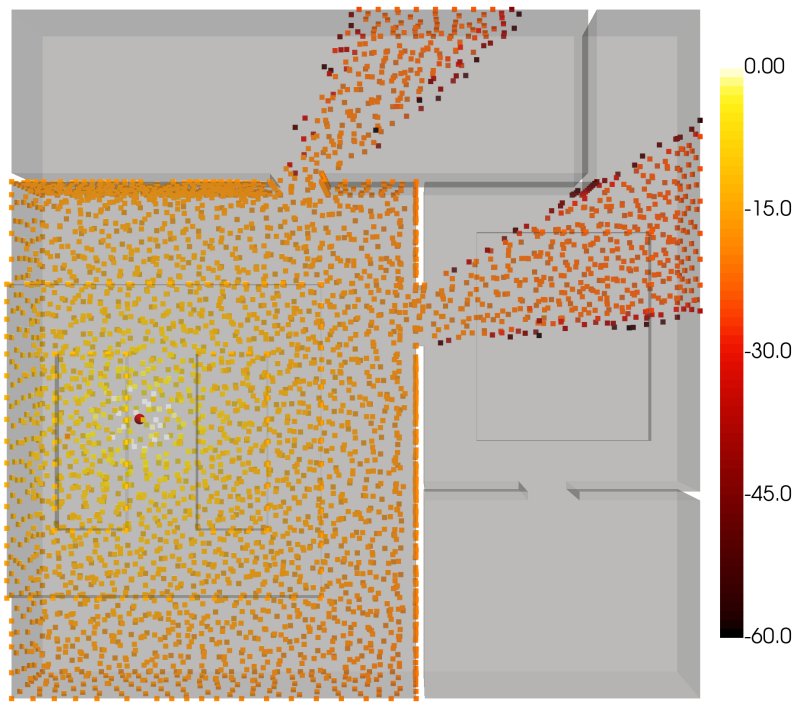
MA in 3D: shadow zones and tapering

What does this look like?

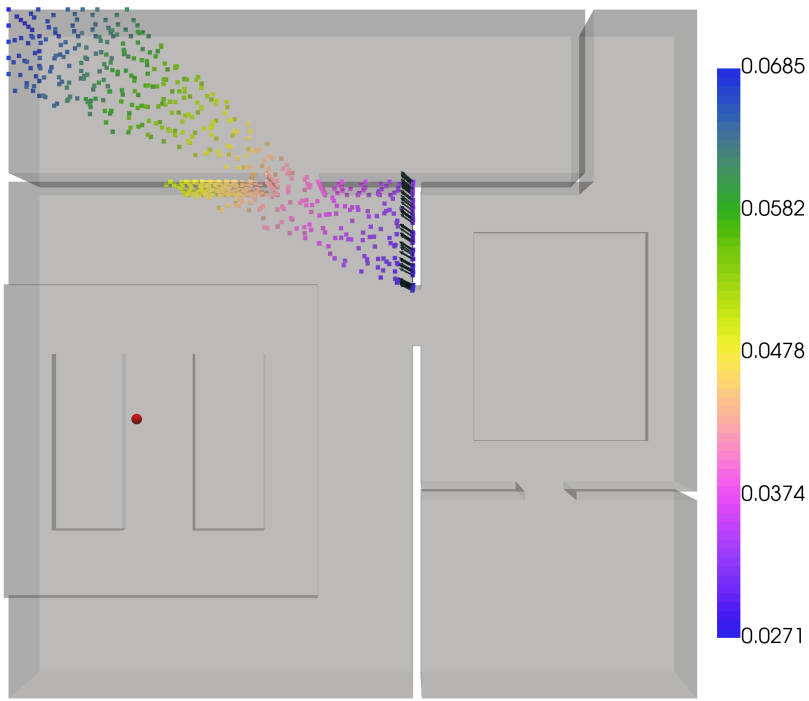


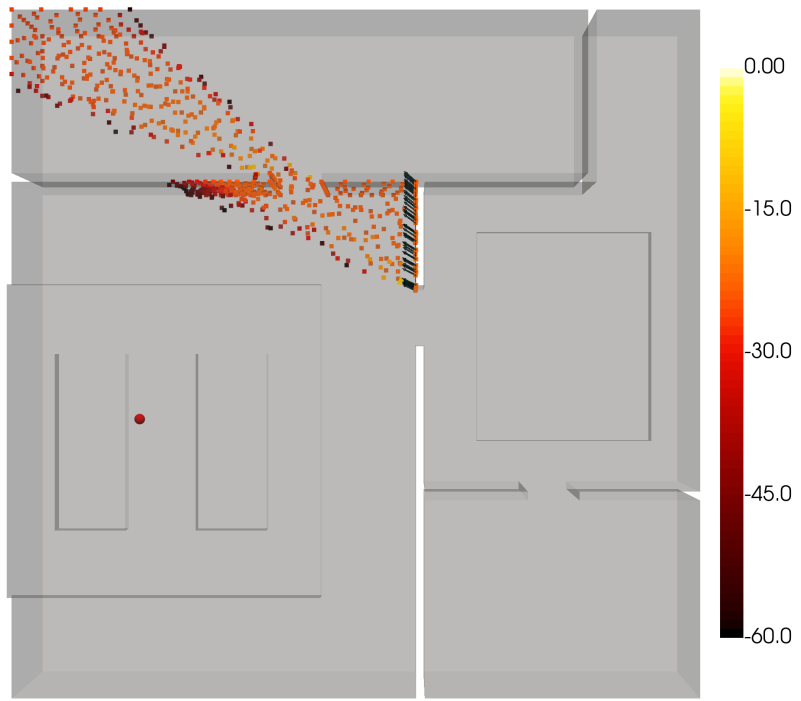
MA in 3D: building test problem



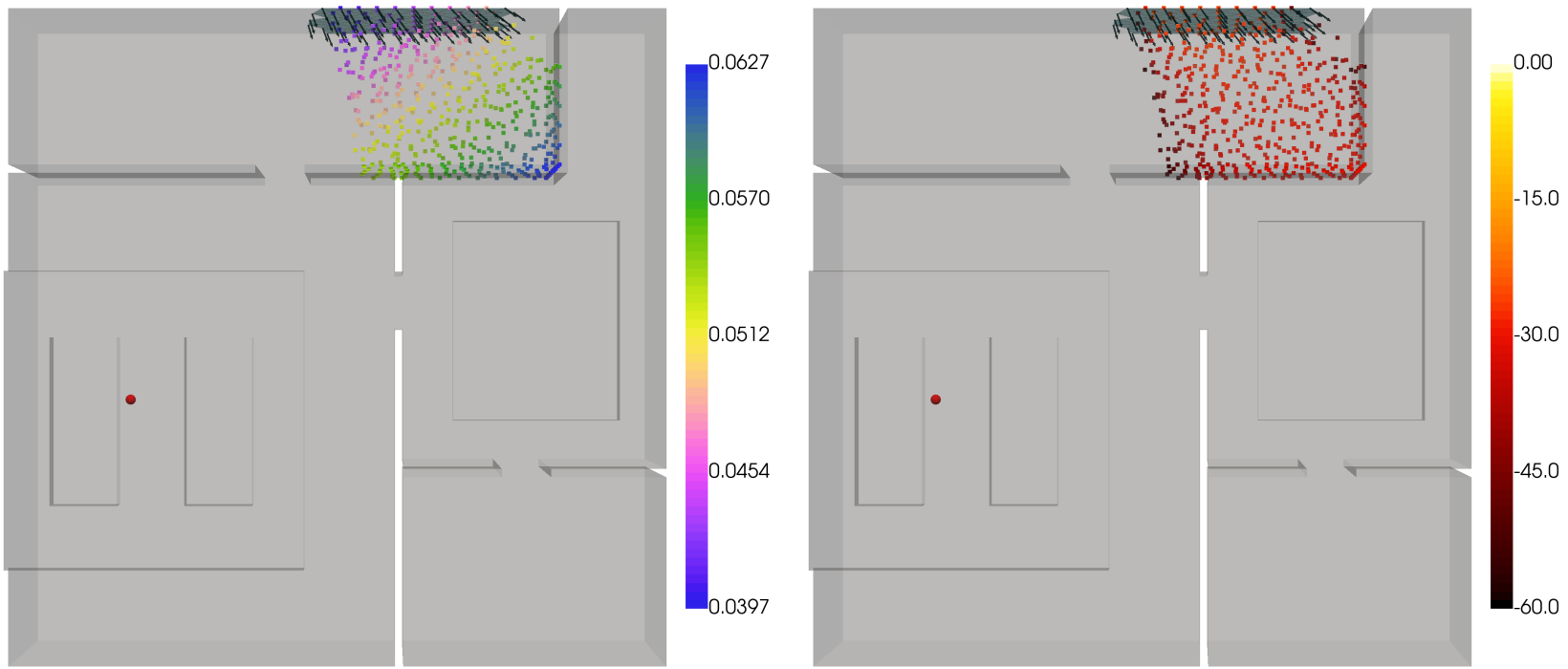


MA in 3D: building test problem

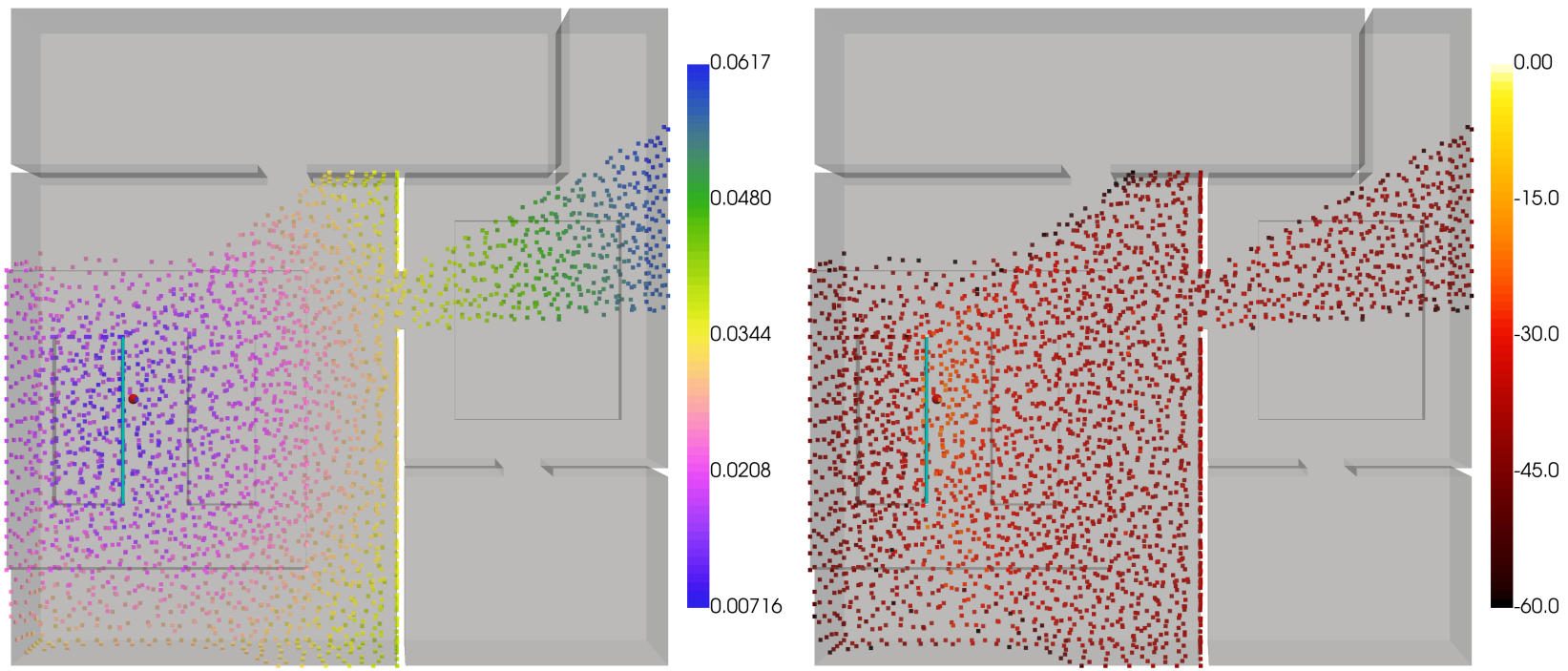




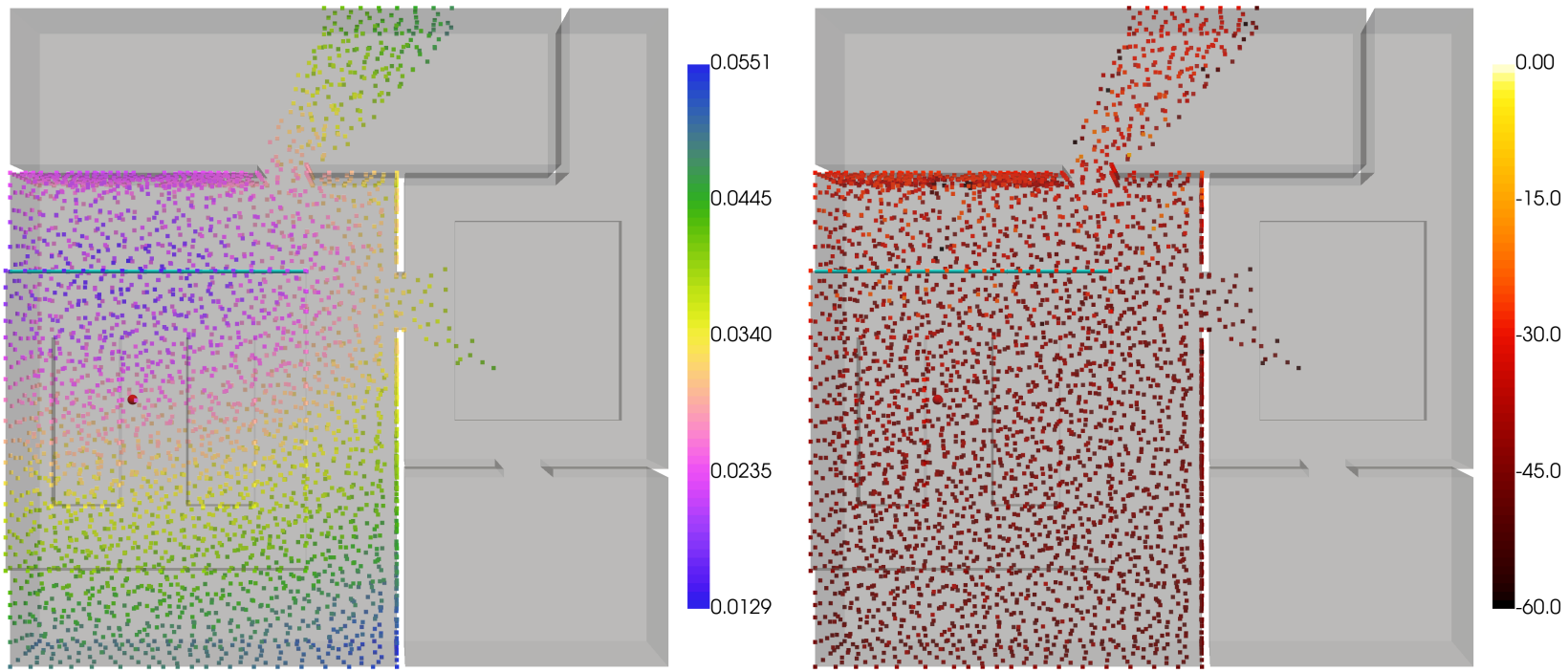
MA in 3D: building test problem



MA in 3D: building test problem



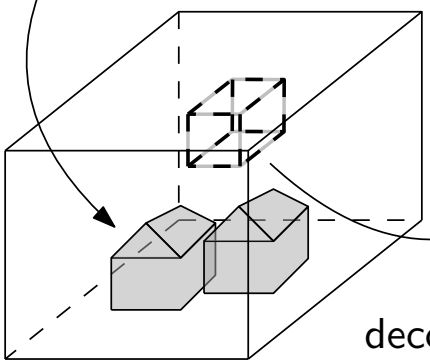
MA in 3D: building test problem



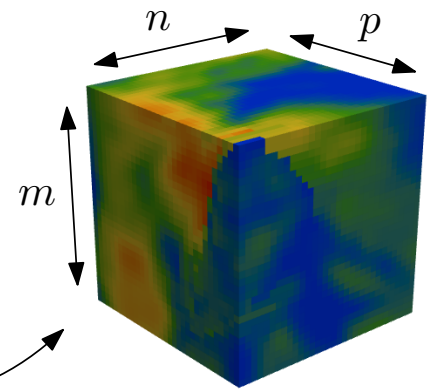
Conclusion and future work

Future work: end-to-end system

biharmonic inpainting to remove geometry



extract subvolume for each channel



collect flattened subvolumes into data matrix X

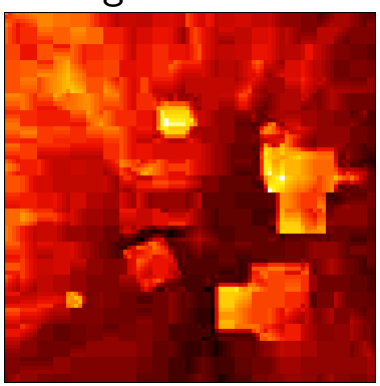
$$X = \begin{bmatrix} x_1 \\ \vdots \\ x_c \end{bmatrix} \in \mathbb{R}^{c \times mnp}$$

$$X = U\Sigma V^\top =: UW^\top$$

$W :=$ "modes" $\in \mathbb{R}^{c \times mnp}$

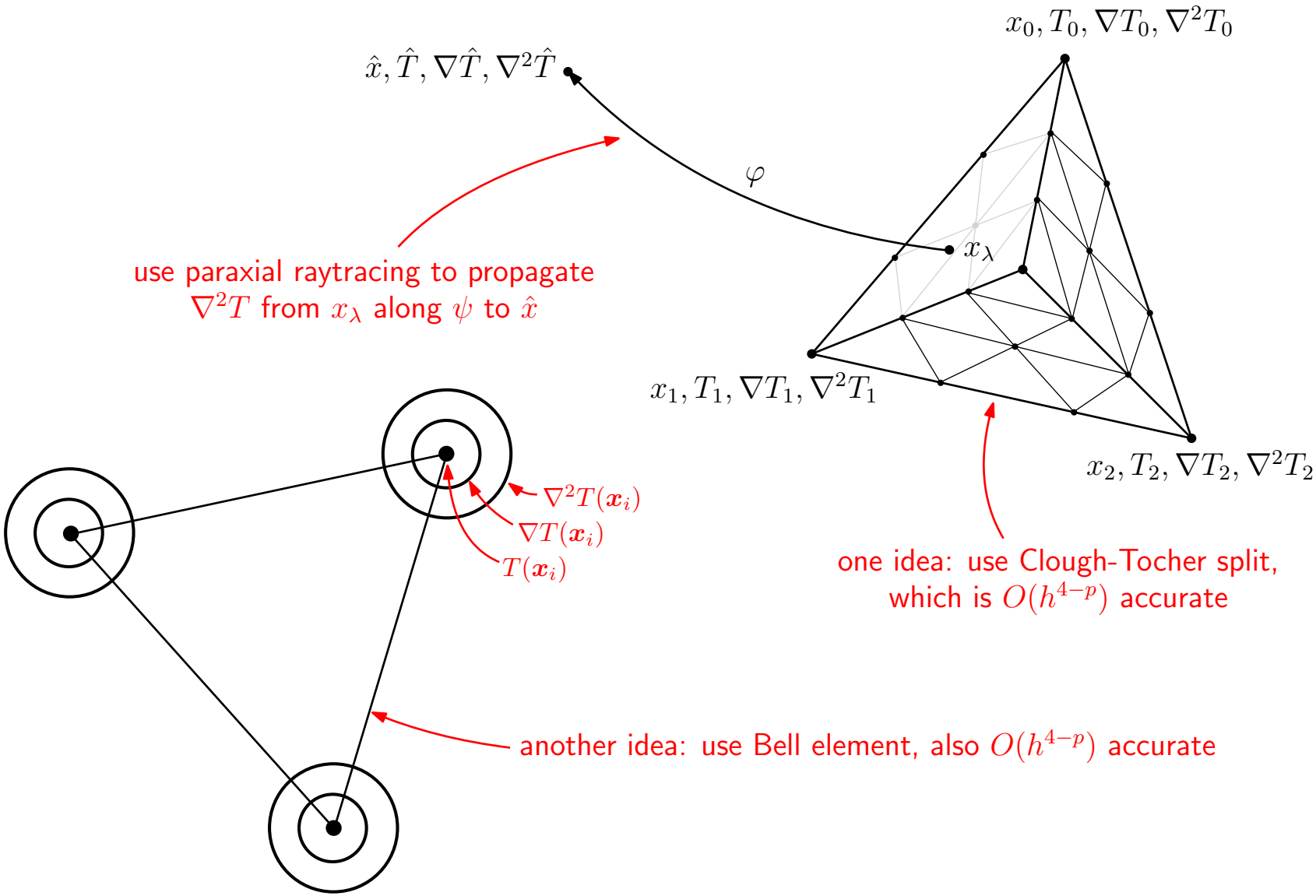
reshape each mode w_i into a $m \times n \times p$ volume, compress using lossy floating-point compression (zfp), optimize to choose compression rate

decode block by block using LRU cache



Future work: $O(h^3)$ in 3D

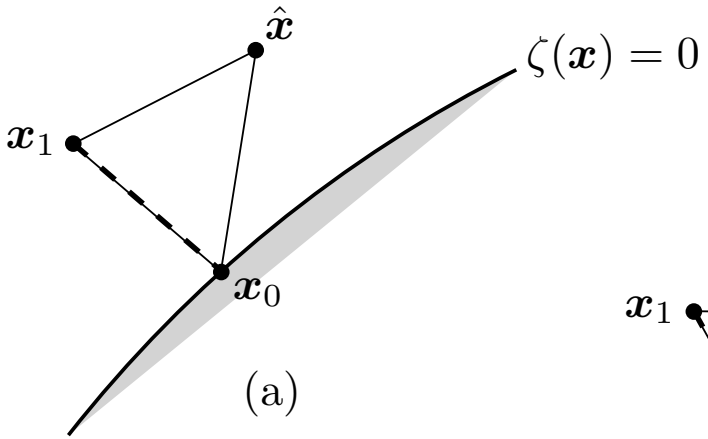
Replace 9-param triangle with **higher-order Hermite element**



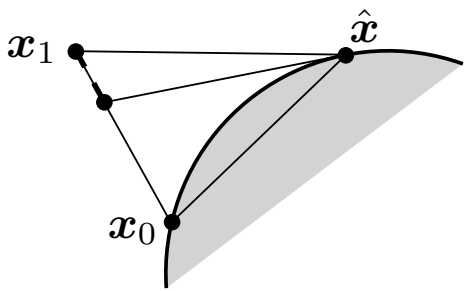
Future work: curved interfaces

Modify the optimization problems solved in the boundary layer to account for **occlusion**

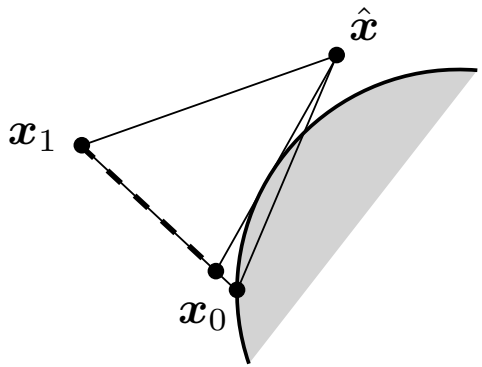
$$\begin{aligned} \text{minimize} \quad & T(\boldsymbol{x}_{\boldsymbol{\lambda}}) + \delta T(\hat{\boldsymbol{x}}, \boldsymbol{x}_{\boldsymbol{\lambda}},) \\ \text{subject to} \quad & \boldsymbol{x}_{\boldsymbol{\lambda}} \in \text{conv}(\boldsymbol{x}_0, \boldsymbol{x}_1, \boldsymbol{x}_2) \\ & \zeta(\boldsymbol{\varphi}(\sigma)) \geq 0. \end{aligned}$$



(a)



(b)

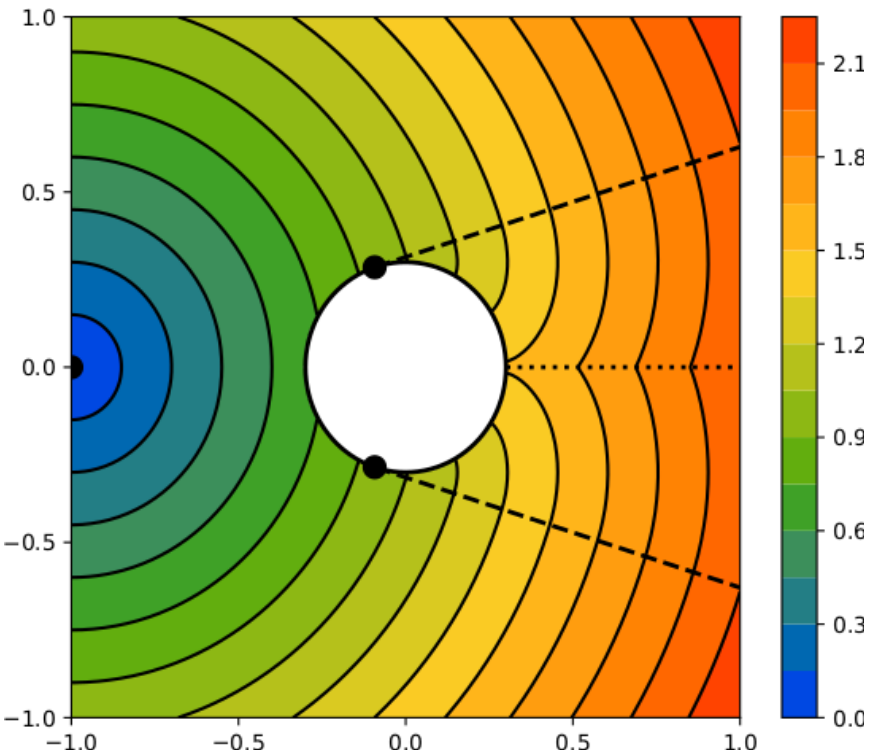


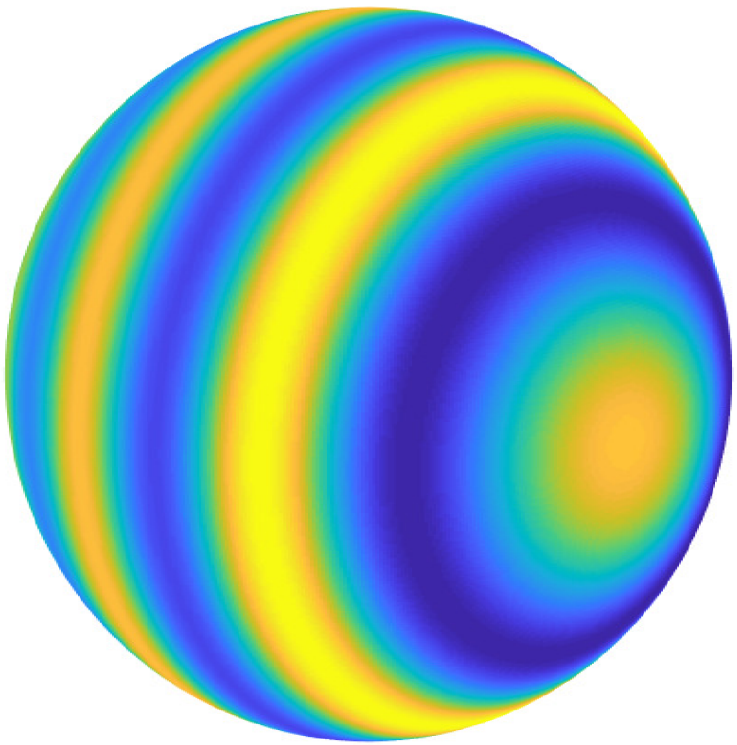
(c)

Future work: scattering from a sphere

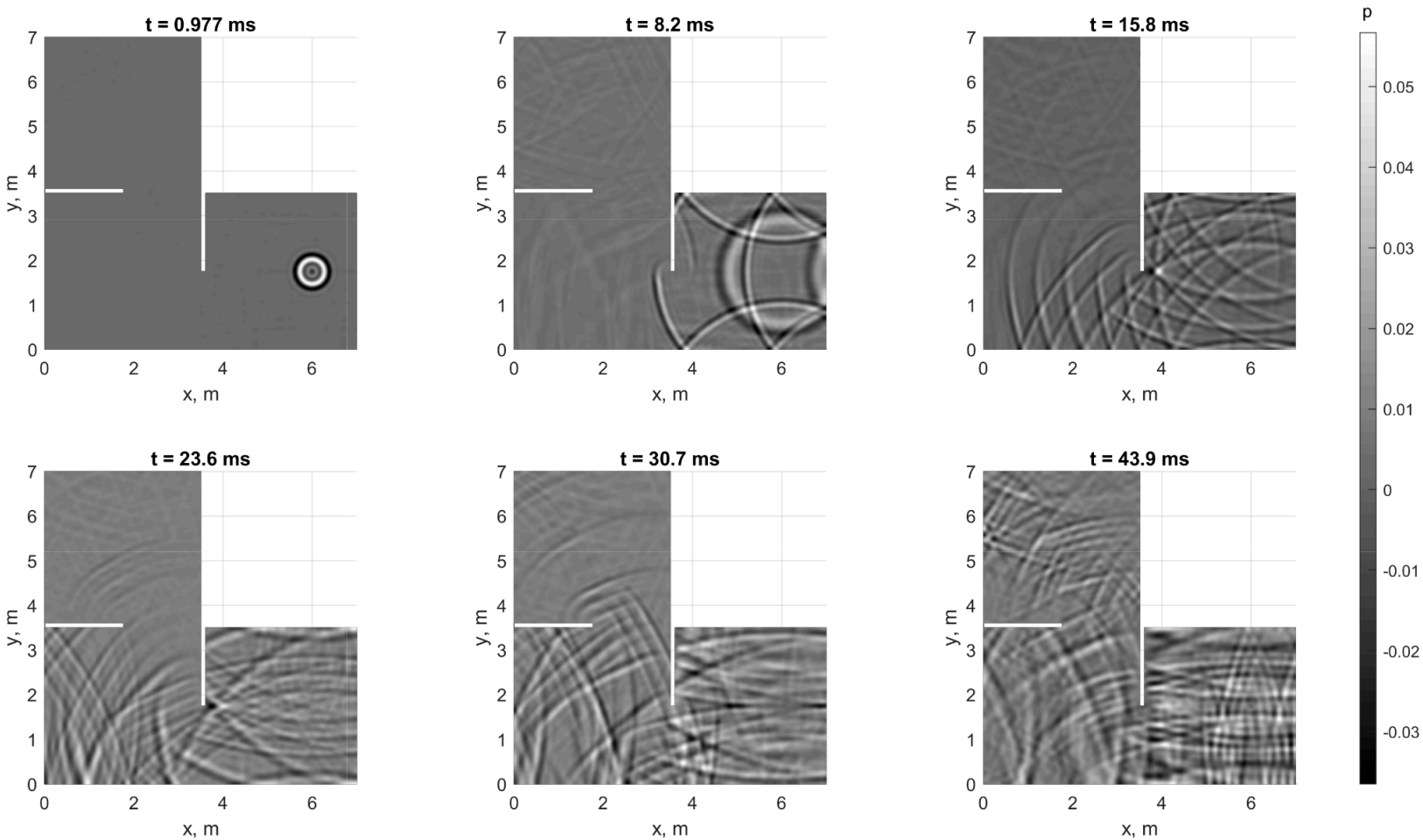
Analytic formula for sound hard scattering from a sphere:

$$P(\boldsymbol{x}) = ik \sum_{n=0}^{\infty} \sum_{m=-n}^{n} \left[j_n(kr) - \frac{j_n'(kr)}{h_n'(kr)} h_n(kr) \right] S_n^{-m}(\boldsymbol{x}_{\text{src}} - \boldsymbol{x}_0)\, Y_n^m(\boldsymbol{s})$$





Future work: comparison with other methods



Gumerov and Duraiswami, 2021.

Acknowledgments

Masha Ramani John Nikunj

